

```
=====
文件名称: Faster-Whisper_AMICorpus_Offline_WER.py
=====
```

```
import os
import sys
import re
import jiwer
from faster_whisper import WhisperModel
# ===== 配置区域 =====
# 本程序用于:
# 使用 faster-whisper 对 AMI Corpus 音频进行自动语音识别 (ASR),
# 并结合参考文本计算词错误率 (WER), 评估模型识别准确性。
# AMI 数据集根目录 (假设 wav 和对应 txt 在同一目录)
AMI_DIR = "/opt/Audio_Datasets/AMICorpus"
# Whisper 模型规格 (medium.en 为英文中等规模模型)
MODEL_SIZE = "medium.en"
# 最小有效单词数限制:
# 少于该阈值的样本容易导致 WER 波动过大, 因此不纳入统计
MIN_WORD_COUNT = 3
# 最大处理文件数限制:
# 用于快速测试或在并发实验中控制样本规模
MAX_FILES = 10
# ===== 辅助函数 =====
def normalize_text(text):
    """
    文本归一化函数 (参考文本 & 识别文本统一处理)
    处理目标:
    - 消除标点、大小写、AMI 特有标签等差异
    - 保证 WER 计算只反映“词级内容差异”, 而非格式差异
    处理步骤:
    1. 将换行符替换为空格 (AMI 文本中常见)
    2. 全部转为小写
    3. 移除 <...> 形式的标注标签 (如 <SIL>、<vocal>)
    4. 移除标点符号, 仅保留字母、数字和空格
    5. 拆分为单词列表
    """
    if not text:
        return []
    # 1. 替换回车与换行为空格, 避免单词被拼接
    text = text.replace("\n", " ")
    text = text.replace("\r", " ")
    # 2. 转为小写, 避免大小写影响 WER
    text = text.lower()
    # 3. 移除 <...> 格式的 AMI 特有标签
    text = re.sub(r"<[^\>]+>", " ", text)
    # 4. 移除除字母、数字、空格外的所有符号
    text = re.sub(r"[^\w\s]", "", text)
    # 5. 按空格切分为单词序列
    words = text.strip().split()
    return words
```

```

# ===== Faster-Whisper 初始化 =====
def create_recognizer(model_size):
    """
    初始化 Faster-Whisper 识别器
    关键配置说明：
    - device="cuda"：使用 GPU 推理
    - compute_type="float16"：半精度计算，加速推理并降低显存占用
    """
    try:
        model = WhisperModel(
            model_size,
            device="cuda",
            compute_type="float16"
        )
        return model
    except Exception as e:
        print(f"模型加载失败: {e}")
        sys.exit(1)

# ===== 主程序 =====
def main():
    print("-" * 50)
    print(f"正在加载模型 {MODEL_SIZE} (AMI Corpus)...")
    # 初始化 ASR 模型
    recognizer = create_recognizer(MODEL_SIZE)
    print("模型加载完成。")
    print(f"注意：程序将在处理完 {MAX_FILES} 个有效文件后停止。")
    print("-" * 50)
    # 检查数据集路径
    if not os.path.exists(AMI_DIR):
        print(f"错误：数据集目录不存在 {AMI_DIR}")
        return
    # 全局统计量（用于最终平均 WER 计算）
    total_distance = 0      # 累计编辑距离 (S + D + I)
    total_ref_words = 0     # 累计参考单词数
    file_count = 0          # 已成功评测的文件数
    skipped_count = 0       # 被跳过的文件数（过短或异常）
    stop_processing = False # 控制提前退出目录遍历
    print(f"开始遍历数据集: {AMI_DIR}")
    # 遍历 AMI 数据集目录结构
    for root, dirs, files in os.walk(AMI_DIR):
        if stop_processing:
            break
        # 仅筛选 wav 音频文件
        wav_files = [f for f in files if f.lower().endswith('.wav')]
        for wav_file in wav_files:
            # 达到最大文件数限制后立即停止
            if file_count >= MAX_FILES:
                stop_processing = True
                break
            wav_path = os.path.join(root, wav_file)

```

```

# 构造对应的参考文本路径
# 规则: audio.wav -> audio.txt
file_id = os.path.splitext(wav_file)[0]
txt_file = file_id + ".txt"
txt_path = os.path.join(root, txt_file)
# 1. 检查是否存在参考文本
if not os.path.exists(txt_path):
    continue
# 2. 读取参考文本
try:
    with open(txt_path, "r", encoding="utf-8") as f:
        raw_ref_text = f.read()
except Exception as e:
    print(f"无法读取文本文件 {txt_path}: {e}")
    continue
# 3. 对参考文本进行归一化
ref_words_list = normalize_text(raw_ref_text)
# 4. 过滤过短样本 (避免 WER 不稳定)
if len(ref_words_list) < MIN_WORD_COUNT:
    skipped_count += 1
    continue
# ===== 核心推理逻辑 =====
try:
    # 使用 faster-whisper 进行转写
    segments, info = recognizer.transcribe(
        wav_path,
        beam_size=5,                    # Beam Search, 提高识别准确率
        language="en",                  # 指定英语
        vad_filter=True,                 # 启用 VAD, 过滤静音
        condition_on_previous_text=True # 启用上下文条件
    )
    # 将所有 segment 的文本拼接为完整假设文本
    hyp_segments_list = [segment.text for segment in segments]
    hyp_text_raw = " ".join(hyp_segments_list)
except Exception as e:
    print(f"推理异常 {wav_file}: {e}")
    continue
# ===== WER 评测逻辑 =====
# 对识别文本进行同样的归一化处理
hyp_words_list = normalize_text(hyp_text_raw)
# 转为字符串形式供 jiwer 使用
ref_str_clean = " ".join(ref_words_list)
hyp_str_clean = " ".join(hyp_words_list)
try:
    # 使用 jiwer 计算词级编辑距离
    out = jiwer.process_words(ref_str_clean, hyp_str_clean)
    # 编辑距离 = 替换 + 删除 + 插入
    curr_dist = out.substitutions + out.deletions + out.insertions
    curr_wer = out.wer

```

```

        curr_len = len(out.references[0]) if out.references else 0
        # 累计全局统计量
        total_distance += curr_dist
        total_ref_words += curr_len
        file_count += 1
        # 输出当前文件评测结果
        print(
            f"[{file_count}/{MAX_FILES}] "
            f"File: {wav_file} | "
            f"WER: {curr_wer:.4f} | "
            f"Dist: {curr_dist} | "
            f"RefWords: {curr_len}"
        )
    except Exception as e:
        print(f"Jiwer 计算错误 {wav_file}: {e}")
# ===== 结果汇总 =====
print("\n" + "=" * 50)
if stop_processing:
    print(f"已达到最大处理数量限制 ({MAX_FILES} 个), 停止运行。")
print("AMI Corpus 评测完成")
print(f"有效处理文件数: {file_count}")
print(f"跳过短/无效文件数: {skipped_count}")
# 计算整体平均 WER
if total_ref_words > 0:
    avg_wer = total_distance / total_ref_words
    print("-" * 25)
    print(f"总编辑距离: {total_distance}")
    print(f"总参考单词数: {total_ref_words}")
    print(f"平均错词率 (Average WER): {avg_wer:.2%}")
else:
    print("未生成有效统计数据。")
print("=" * 50)
if __name__ == "__main__":
    main()

```

```

=====
文件名称: Faster-Whisper_AMICorpus_Slide_WER.py
=====

```

```

import os
import sys
import re
import wave
import numpy as np
import jiwer
from faster_whisper import WhisperModel
# ===== 配置区域 =====
# 本程序用于:
# 使用 Faster-Whisper (large-v3) 对 AMI Corpus 音频进行语音识别,
# 并通过参考文本计算 WER, 同时统计音频时长、编辑距离等指标。

```

```

# AMI 数据集根目录（包含 wav 与对应 txt）
AMICORPUS_DIR = "/opt/Audio_Datasets/AMICorpus"
# Whisper 模型规格（large-v3：高精度模型）
MODEL_SIZE = "large-v3"
# 推理设备与精度配置
DEVICE = "cuda"          # 使用 GPU
COMPUTE_TYPE = "float16" # 半精度计算，加速推理、降低显存占用
# 仅处理前 N 个文件（便于快速评测或调试）
MAX_FILES_TO_PROCESS = 10
# ===== 辅助函数 =====
def get_audio_duration(file_path):
    """
    获取 wav 音频时长（单位：秒）
    说明：
    - 通过 wave 模块读取帧数与采样率计算
    - 用于输出结果表格中的 Duration 字段
    """
    try:
        with wave.open(file_path, 'rb') as wf:
            frames = wf.getnframes()
            rate = wf.getframerate()
            return frames / float(rate)
    except Exception:
        return 0.0
def normalize_text(text):
    """
    文本归一化函数（用于参考文本与识别文本）
    处理规则：
    1. 将回车、换行替换为空格（AMI 文本常见）
    2. 转为小写，消除大小写差异
    3. 去除标点，仅保留：
        - 字母
        - 数字
        - 空格
        - 撇号（用于 don't, I'm 等缩写）
    4. 拆分为单词列表，用于 WER 计算
    """
    if not text:
        return []
    # 替换回车/换行为空格，避免单词粘连
    text = text.replace('\n', ' ').replace('\r', ' ')
    # 转为小写
    text = text.lower()
    # 去除除字母、数字、空格、撇号外的所有字符
    text = re.sub(r"[^a-z0-9\s']+", "", text)
    # 按空格分词
    words = text.strip().split()
    return words
# ===== 核心识别函数（智能截断 + 防幻觉） =====
def transcribe_wav_file(model, file_path, verbose=False):

```

"""

对单个 wav 文件进行语音识别（模拟流式处理）

核心特点：

- 使用 wave 分块读取音频，模拟流式输入
- 引入“智能截断”策略，避免重复输出
- 通过多规则过滤 Whisper 幻觉（hallucination）

"""

# 单次处理的音频块长度（秒）

CHUNK\_DURATION = 1.0

# 采样率（AMI Corpus 通常为 16kHz）

SAMPLE\_RATE = 16000

# 强制提交最长等待时间（防止长时间缓存）

FORCE\_FLUSH\_DURATION = 10.0

# 常见 Whisper 幻觉关键词黑名单

HALLUCINATION\_PHRASES = ["subtitle", "amara", "audio", "copyright", "subscribe"]

def is\_hallucination(segment):

"""

幻觉检测规则：

- 空文本或无有效时长
- 语速异常快
- 单词密度异常
- 高静音概率
- 平均对数概率过低
- 命中黑名单关键词

"""

text = segment.text.strip().lower()

duration = segment.end - segment.start

if not text or duration <= 0:

return True, "Empty"

if (len(text) / duration) > 40:

return True, "Too Fast"

if segment.words and len(segment.words) > 1:

if (duration / len(segment.words)) < 0.05:

return True, "Word Density"

if segment.no\_speech\_prob > 0.9:

return True, "Silence Prob"

if segment.avg\_logprob < -1.5:

return True, "Low Logprob"

for bad in HALLUCINATION\_PHRASES:

if bad in text:

return True, "Blacklist"

return False, "OK"

# 音频缓存区（浮点格式）

audio\_buffer = np.array([], dtype=np.float32)

# 最终输出文本片段列表

final\_transcript = []

# 打开 wav 文件

try:

wf = wave.open(file\_path, 'rb')

# AMI 音频通常为 16kHz 单声道，这里做校验

```

        if wf.getnchannels() != 1 or wf.getframerate() != 16000:
            return f"Error: Format mismatch ({wf.getnchannels()}ch {wf.getframerate()}Hz). Expect
1ch 16000Hz"
    except Exception as e:
        return f"Error opening wav: {e}"
# 每次读取的采样点数
chunk_samples = int(SAMPLE_RATE * CHUNK_DURATION)
while True:
    # 从 wav 中读取音频块
    data = wf.readframes(chunk_samples)
    is_eof = len(data) == 0
    # 非 EOF: 追加到缓存
    if not is_eof:
        audio_int16 = np.frombuffer(data, dtype=np.int16)
        new_chunk = audio_int16.astype(np.float32) / 32768.0
        audio_buffer = np.concatenate((audio_buffer, new_chunk))
    # EOF 且缓存为空: 结束
    if is_eof and len(audio_buffer) == 0:
        break
    buffer_duration = len(audio_buffer) / SAMPLE_RATE
    # EOF 且剩余音频过短, 直接结束
    if is_eof and buffer_duration < 0.2:
        break
    # 非 EOF 且缓存太短, 继续累积
    if buffer_duration < 0.5 and not is_eof:
        continue
    # 调用 Whisper 进行识别
    segments, _ = model.transcribe(
        audio_buffer,
        beam_size=1,
        language="en",
        condition_on_previous_text=False,
        vad_filter=True,
        vad_parameters=dict(min_silence_duration_ms=300),
        word_timestamps=True,
        no_speech_threshold=0.5
    )
    # 收集通过幻觉过滤的单词
    all_valid_words = []
    for seg in segments:
        is_bad, reason = is_hallucination(seg)
        if is_bad:
            continue
        # 若未返回词级时间戳, 则构造伪 word
        if not seg.words:
            class PseudoWord:
                def __init__(self, w, start, end):
                    self.word = w
                    self.start = start
                    self.end = end

```

```

        all_valid_words.append(PseudoWord(seg.text, seg.start, seg.end))
    else:
        all_valid_words.extend(seg.words)
word_count = len(all_valid_words)
# 是否需要保留末尾词（避免过早提交）
should_hold_back = (not is_eof) and (buffer_duration < FORCE_FLUSH_DURATION)
text_to_commit = ""
cut_time = 0.0
# 智能截断逻辑
if word_count > 0:
    if should_hold_back:
        num_to_hold = 1
        last_word_text = all_valid_words[-1].word.strip()
        if last_word_text in ['-', '—', '—']:
            num_to_hold = 2
        if word_count > num_to_hold:
            cut_index = -num_to_hold
            cut_time = all_valid_words[cut_index - 1].end
            committed_words = all_valid_words[:cut_index]
            text_to_commit = "".join([w.word for w in committed_words])
        else:
            cut_time = all_valid_words[-1].end
            text_to_commit = "".join([w.word for w in all_valid_words])
# 提交文本
if text_to_commit:
    final_transcript.append(text_to_commit)
# 从缓存中裁剪已提交部分
if cut_time > 0:
    cut_sample_index = int(cut_time * SAMPLE_RATE)
    if cut_sample_index < len(audio_buffer):
        audio_buffer = audio_buffer[cut_sample_index:]
    else:
        audio_buffer = np.array([], dtype=np.float32)
if is_eof:
    break
wf.close()
return "".join(final_transcript).strip()
# ===== 主程序 =====
def main():
    print("-" * 50)
    print(f"Loading Model: {MODEL_SIZE} on {DEVICE}...")
    # 初始化 Whisper 模型
    try:
        model = WhisperModel(MODEL_SIZE, device=DEVICE, compute_type=COMPUTE_TYPE)
    except Exception as e:
        print(f"模型加载失败: {e}")
        return
    print("Model Loaded.")
    print("-" * 50)
    # 扫描 wav/txt 文件对（最多 MAX_FILES_TO_PROCESS 个）

```



```

print(f"Scanning {AMICORPUS_DIR} for first {MAX_FILES_TO_PROCESS} file pairs...")
file_pairs = []
for root, dirs, files in os.walk(AMICORPUS_DIR):
    for f in files:
        if f.endswith(".wav"):
            wav_path = os.path.join(root, f)
            txt_path = os.path.join(root, os.path.splitext(f)[0] + ".txt")
            if os.path.exists(txt_path):
                file_pairs.append((wav_path, txt_path))
                if len(file_pairs) >= MAX_FILES_TO_PROCESS:
                    break
    if len(file_pairs) >= MAX_FILES_TO_PROCESS:
        break
if not file_pairs:
    print("未找到任何成对的 wav/txt 文件。")
    return
# 输出表头
print("=" * 100)
print(f"{'Filename':<30} | {'Duration':<8} | {'RefWords':<8} | {'Dist':<6} | {'WER':<8}")
print("-" * 100)
total_distance = 0
total_ref_words = 0
processed_count = 0
for wav_path, txt_path in file_pairs:
    file_id = os.path.basename(wav_path)
    # 获取音频时长
    duration = get_audio_duration(wav_path)
    # 读取参考文本
    try:
        with open(txt_path, 'r', encoding='utf-8') as f:
            raw_ref_text = f.read().replace('\n', ' ').replace('\r', ' ')
    except Exception as e:
        print(f"{file_id:<30} | Error reading txt: {e}")
        continue
    # 执行语音识别
    hyp_text_raw = transcribe_wav_file(model, wav_path)
    # 文本归一化
    ref_words = normalize_text(raw_ref_text)
    hyp_words = normalize_text(hyp_text_raw)
    if not ref_words:
        continue
    # WER 计算
    out = jiwer.process_words(" ".join(ref_words), " ".join(hyp_words))
    dist = out.substitutions + out.deletions + out.insertions
    total_distance += dist
    total_ref_words += len(ref_words)
    processed_count += 1
    # 输出结果行
    print(f"{file_id:<30} | {duration:<8.2f} | {len(ref_words):<8} | {dist:<6} | {out.wer:<8.4f}")

```

```

# ===== 汇总 =====
print("=" * 100)
print(f"Processed Files: {processed_count}")
if total_ref_words > 0:
    avg_wer = total_distance / total_ref_words
    print(f"Total Distance: {total_distance}")
    print(f"Total Ref Words: {total_ref_words}")
    print("-" * 30)
    print(f"Average WER: {avg_wer:.2%}")
else:
    print("No valid data for WER calculation.")
print("=" * 100)
if __name__ == "__main__":
    main()

```

```

=====
文件名称: Faster-Whisper_AMICorpus_WER.py
=====

```

```

import os
import sys
import re
import soundfile as sf
import numpy as np
from faster_whisper import WhisperModel
import jiwer
# ===== 配置区域 =====
# AMI 语音数据集根目录（包含 audio/ 与对应转写 txt）
AMI_ROOT = "/opt/Audio_Datasets/AMICorpus"
# 限制最多处理的音频文件数量（用于快速评测）
MAX_FILES_TO_PROCESS = 10
# Whisper 模型规格
MODEL_SIZE = "medium.en"
# 音频分片时长（秒）
# 这里采用 1 秒滑动窗口（非重叠）
CHUNK_DURATION = 1.0
# 静音判断阈值（基于 RMS 能量）
SILENCE_THRESHOLD = 0.01
# 幻觉熔断阈值：
# 若 1 秒音频中识别出的单词数超过该值，视为异常输出
MAX_WORDS_PER_SEC = 6
# ===== 辅助函数 =====
def normalize_text(text):
    """
    对文本进行统一归一化处理，用于 WER 计算：
    1. 替换换行符，避免单词拼接
    2. 转为小写
    3. 移除标点符号
    4. 按空格切分为词序列
    """

```

```

    if not text:
        return []
    text = text.replace('\n', ' ').replace('\r', ' ')
    text = text.lower()
    text = re.sub(r"[\w\s]", "", text)
    words = text.strip().split()
    return words
def calculate_rms(audio_chunk):
    """
    计算音频分片的 RMS（均方根）能量，
    用于简单能量型 VAD（静音检测）
    """
    if len(audio_chunk) == 0:
        return 0
    return np.sqrt(np.mean(audio_chunk ** 2))
def get_ami_file_pairs(root_dir, limit=10):
    """
    遍历 AMI 数据集目录结构，
    在 audio 目录下查找 wav / txt 成对文件
    返回：
    [
        (wav_path, txt_path),
        ...
    ]
    """
    pairs = []
    print(f"正在搜索 {root_dir} 下的前 {limit} 个有效文件对...")
    for root, dirs, files in os.walk(root_dir):
        # AMI 语音文件位于 audio 子目录
        if os.path.basename(root) != 'audio':
            continue
        for file in files:
            if file.endswith(".wav"):
                wav_path = os.path.join(root, file)
                txt_name = os.path.splitext(file)[0] + ".txt"
                txt_path = os.path.join(root, txt_name)
                # 只处理存在官方转写的音频
                if os.path.exists(txt_path):
                    pairs.append((wav_path, txt_path))
                    # 达到限制数量即停止
                    if len(pairs) >= limit:
                        return pairs
    return pairs
# ===== Faster-Whisper 初始化 =====
def create_recognizer(model_size):
    """
    初始化 Faster-Whisper 推理模型
    使用 GPU + FP16 加速
    """
    try:

```

```

        model = WhisperModel(
            model_size,
            device="cuda",
            compute_type="float16"
        )
    return model
except Exception as e:
    print(f"模型加载失败: {e}")
    sys.exit(1)
# ===== 主程序 =====
def main():
    print("-" * 50)
    print(f"正在加载模型 {MODEL_SIZE} (AMI Corpus 适配版)...")
    recognizer = create_recognizer(MODEL_SIZE)
    print("模型加载完成。")
    print("-" * 50)
    # 数据集路径检查
    if not os.path.isdir(AMI_ROOT):
        print(f"错误: 数据集目录不存在: {AMI_ROOT}")
        return
    # 1. 获取待测 wav / txt 文件对
    file_pairs = get_ami_file_pairs(
        AMI_ROOT,
        limit=MAX_FILES_TO_PROCESS
    )
    if not file_pairs:
        print("未找到任何成对的 wav/txt 文件。")
        return
    print(f"已找到 {len(file_pairs)} 个待测文件。")
    print("-" * 50)
    # WER 累计量
    total_distance = 0
    total_ref_words = 0
    processed_count = 0
    # ===== 文件级评测循环 =====
    for wav_path, txt_path in file_pairs:
        # --- A. 读取并清洗官方参考文本 ---
        try:
            with open(txt_path, 'r', encoding='utf-8') as f:
                raw_text = f.read()
            ref_words_list = normalize_text(raw_text)
            if len(ref_words_list) == 0:
                print(f"警告: 文本为空, 跳过 {os.path.basename(wav_path)}")
                continue
        except Exception as e:
            print(f"读取文本失败 {txt_path}: {e}")
            continue
        # --- B. 读取音频 ---
        try:
            audio, sample_rate = sf.read(

```

```

        wav_path,
        dtype="float32"
    )
except Exception as e:
    print(f"读取音频失败 {wav_path}: {e}")
    continue
# 音频总时长 (秒)
duration_sec = len(audio) / sample_rate
# 忽略极短音频 (防止异常统计)
if len(audio) < int(sample_rate * 0.05):
    continue
# ===== 核心推理逻辑 =====
# 使用固定长度滑动窗口 (无重叠) 进行分片推理
chunk_size = int(sample_rate * CHUNK_DURATION)
hyp_segments_list = []
last_text = ""
for i in range(0, len(audio), chunk_size):
    # 取 1 秒音频分片
    chunk = audio[i: i + chunk_size]
    # 极短分片直接跳过
    if len(chunk) < 160:
        continue
    # 1 能量型 VAD: 过滤静音
    rms = calculate_rms(chunk)
    if rms < SILENCE_THRESHOLD:
        continue
    # 2 Whisper 推理
    try:
        segments, info = recognizer.transcribe(
            chunk,
            beam_size=1,                # 低延迟解码
            language="en",
            vad_filter=True,            # Whisper 内置 VAD
            condition_on_previous_text=False,
            temperature=0.0,
            compression_ratio_threshold=2.0,
            no_speech_threshold=0.4
        )
        current_chunk_text = ""
        for segment in segments:
            txt = segment.text.strip()
            # 3 幻觉黑名单过滤
            hallucinations = [
                "see you",
                "watching",
                "subtitles",
                "amara",
                "org",
                "bye"
            ]

```

```

        if any(h in txt.lower() for h in hallucinations):
            continue
        current_chunk_text += " " + txt
        current_chunk_text = current_chunk_text.strip()
        # 4 字数熔断（防止异常爆词）
        words_in_chunk = len(current_chunk_text.split())
        if words_in_chunk > MAX_WORDS_PER_SEC:
            continue
        # 5 防重复输出（相邻窗口重复文本）
        if current_chunk_text and current_chunk_text != last_text:
            hyp_segments_list.append(current_chunk_text)
            last_text = current_chunk_text
    except Exception:
        # 单个分片失败不影响整体评测
        continue
# 拼接所有分片识别结果
hyp_text_raw = " ".join(hyp_segments_list)
# ===== WER 计算 =====
hyp_words_list = normalize_text(hyp_text_raw)
ref_str_clean = " ".join(ref_words_list)
hyp_str_clean = " ".join(hyp_words_list)
try:
    out = jiwer.process_words(
        ref_str_clean,
        hyp_str_clean
    )
    # 编辑距离
    curr_dist = (
        out.substitutions +
        out.deletions +
        out.insertions
    )
    curr_wer = out.wer
    curr_len = len(out.references[0]) if out.references else 0
    total_distance += curr_dist
    total_ref_words += curr_len
    processed_count += 1
    # 文件级输出
    print(
        f"File: {os.path.basename(wav_path)} | "
        f"WER: {curr_wer:.4f} | "
        f"Dist: {curr_dist} | "
        f"RefWords: {curr_len} | "
        f"Duration: {duration_sec:.2f}s"
    )
except Exception as e:
    print(
        f"Jiwer 计算错误 "
        f"{os.path.basename(wav_path)}: {e}"
    )

```

```

# ===== 汇总结果 =====
print("\n" + "=" * 50)
print("评测完成 (AMI Corpus Top 10)")
print(f"实际处理文件数: {processed_count}")
if total_ref_words > 0:
    avg_wer = total_distance / total_ref_words
    print(f"总编辑距离: {total_distance}")
    print(f"总参考单词数: {total_ref_words}")
    print("-" * 25)
    print(f"平均错词率 (Average WER): {avg_wer:.2%}")
else:
    print("未生成任何有效统计数据。")
print("=" * 50)
if __name__ == "__main__":
    main()

```

```

=====
文件名称: Faster-Whisper_LibriSpeech_Offline_WER.py
=====

```

```

import os
import sys
import re
import jiwer
from faster_whisper import WhisperModel
# ===== 配置区域 =====
# LibriSpeech 测试集路径 (test-clean, 已转为 wav 或 flac)
LIBRISPEECH_DIR = "/opt/Audio_Datasets/LibriSpeech_WAV/test-clean"
# Faster-Whisper 模型规格
MODEL_SIZE = "medium.en"
# ===== 辅助函数 =====
def normalize_text(text):
    """
    文本归一化函数，用于 WER 计算前的统一处理：
    1. 转为小写
    2. 去除标点符号
    3. 仅保留单词和空格
    4. 按空格切分为词序列
    """
    if not text:
        return []
    # 转小写
    text = text.lower()
    # 去除标点符号（保留字母、数字和空格）
    text = re.sub(r"[^\w\s]", "", text)
    # 按空格切分为单词列表
    words = text.strip().split()
    return words
# ===== Faster-Whisper 初始化 =====
def create_recognizer(model_size):

```

```

"""
创建 Faster-Whisper 识别器实例：
- 使用 GPU (cuda) 进行推理
- 使用 float16 精度以提升推理性能
"""

try:
    model = WhisperModel(
        model_size,
        device="cuda",
        compute_type="float16"
    )
    return model
except Exception as e:
    print(f"模型加载失败: {e}")
    sys.exit(1)

# ===== 主程序 =====
def main():
    print("-" * 50)
    print(f"正在加载模型 {MODEL_SIZE} (全量音频推理模式)...")
    # 初始化识别模型
    recognizer = create_recognizer(MODEL_SIZE)
    print("模型加载完成。")
    print("-" * 50)
    # 全局统计量 (用于计算平均 WER)
    total_distance = 0      # 累计编辑距离
    total_ref_words = 0     # 累计参考文本单词数
    file_count = 0         # 实际处理音频文件数量
    # 检查数据集路径
    if not os.path.exists(LIBRISPEECH_DIR):
        print(f"错误: 数据集目录不存在 {LIBRISPEECH_DIR}")
        return
    print(f"开始遍历数据集: {LIBRISPEECH_DIR}")
    # ===== 数据集遍历 =====
    for root, dirs, files in os.walk(LIBRISPEECH_DIR):
        # LibriSpeech 中每个目录包含一个 *.trans.txt 文件
        trans_files = [f for f in files if f.endswith(".trans.txt")]
        for trans_file in trans_files:
            trans_path = os.path.join(root, trans_file)
            ref_map = {}
            # ----- 读取官方转写文本 -----
            # trans.txt 格式:
            # <file_id> <reference text>
            try:
                with open(trans_path, "r", encoding="utf-8") as f:
                    for line in f:
                        parts = line.strip().split(" ", 1)
                        if len(parts) == 2:
                            file_id, text = parts
                            ref_map[file_id] = text
            except Exception as e:

```



```

        print(f"读取文本文件失败 {trans_path}: {e}")
        continue
# ----- 遍历该目录下的音频与参考文本 -----
for file_id, ref_text in ref_map.items():
    # 优先查找 wav 文件
    audio_filename = f"{file_id}.wav"
    audio_path = os.path.join(root, audio_filename)
    # 若 wav 不存在, 则尝试 flac
    if not os.path.exists(audio_path):
        flac_path = os.path.join(root, f"{file_id}.flac")
        if os.path.exists(flac_path):
            audio_path = flac_path
        else:
            # 未找到对应音频文件, 跳过
            continue
# ===== 核心推理逻辑 (全量音频) =====
try:
    """
    与滑动窗口方案不同:
    - 这里直接对完整音频进行一次性推理
    - Whisper 内部自行进行分段
    - 更适合离线准确率评测
    """

    segments, info = recognizer.transcribe(
        audio_path,
        beam_size=5,                    # Beam Search, 提高识别准确率
        language="en",
        vad_filter=True,                # 使用内置 VAD 过滤非语音片段
        vad_parameters=dict(
            min_silence_duration_ms=500
        ),
        condition_on_previous_text=True # 允许使用前文上下文 (有利于长句识别)
    )
    # 拼接所有识别出的文本片段
    hyp_segments_list = [
        segment.text for segment in segments
    ]
    hyp_text_raw = " ".join(hyp_segments_list)
except Exception as e:
    print(f"推理失败 {audio_path}: {e}")
    continue
# =====
# ===== WER 评测逻辑 =====
# 对参考文本和识别文本使用同一归一化规则
ref_words_list = normalize_text(ref_text)
hyp_words_list = normalize_text(hyp_text_raw)
ref_str_clean = " ".join(ref_words_list)
hyp_str_clean = " ".join(hyp_words_list)
try:
    # 使用 jiwer 计算词错误率 (WER)

```

```

        out = jiwer.process_words(
            ref_str_clean,
            hyp_str_clean
        )
        # 编辑距离 = 替换 + 删除 + 插入
        curr_dist = (
            out.substitutions +
            out.deletions +
            out.insertions
        )
        curr_wer = out.wer
        curr_len = (
            len(out.references[0])
            if out.references else 0
        )
        # 累计统计
        total_distance += curr_dist
        total_ref_words += curr_len
        file_count += 1
        # 文件级输出
        print(
            f"File: {os.path.basename(audio_path)} | "
            f"WER: {curr_wer:.4f} | "
            f"Dist: {curr_dist} | "
            f"RefWords: {curr_len}"
        )
    except Exception as e:
        print(f"Jiwer 计算错误 {file_id}: {e}")
# ===== 汇总统计结果 =====
print("\n" + "=" * 50)
print("评测完成")
print(f"处理文件总数: {file_count}")
if total_ref_words > 0:
    avg_wer = total_distance / total_ref_words
    print(f"总编辑距离: {total_distance}")
    print(f"总参考单词数: {total_ref_words}")
    print("-" * 25)
    print(f"平均错词率 (Average WER): {avg_wer:.2%}")
else:
    print("未处理任何有效文件。")
    print("=" * 50)
if __name__ == "__main__":
    main()

```

```

=====
文件名称: Faster-Whisper_LibriSpeech_Parallel_RTF.py
=====

```

```

import os
import sys

```

```

import re
import time
import soundfile as sf
import numpy as np
import threading
import queue
from faster_whisper import WhisperModel
import jiwer

# ===== 配置区域 =====
# LibriSpeech 测试集目录 (test-clean)
LIBRISPEECH_DIR = "/opt/Audio_Datasets/LibriSpeech_WAV/test-clean"
# 使用的 Whisper 模型规格
MODEL_SIZE = "medium.en"
# 音频切片长度 (秒), 用于模拟流式/分段推理
CHUNK_DURATION = 1.0 # 1 秒一片
# 并行线程数量 (并发请求数)
PARALLEL_NUM = 5
# ===== VAD & 后处理参数 =====
# 静音能量阈值 (RMS)
SILENCE_THRESHOLD = 0.01
# 字数熔断: 1 秒内输出超过该词数, 认为是幻觉
MAX_WORDS_PER_SEC = 6
# 最小参考词数 (过短样本不参与 WER 统计)
MIN_WORD_COUNT = 3
# ===== 全局共享资源 =====
# 打印锁 (防止多线程 print 交错)
print_lock = threading.Lock()
# 统计锁 (保护全局统计变量)
stats_lock = threading.Lock()
# ===== 全局统计量 (跨线程累加) =====
# 所有线程的推理耗时之和 (秒)
g_total_proc_time = 0.0
# 所有音频的总时长 (秒)
g_total_audio_dur = 0.0
# 参考文本总词数 (用于平均 WER)
g_total_ref_words = 0
# 累积编辑距离
g_total_distance = 0
# ===== 辅助函数 =====
def normalize_text(text):
    """
    文本归一化, 用于 WER 计算:
    - 转小写
    - 去标点
    - 分词
    """
    if not text:
        return []
    text = text.lower()
    text = re.sub(r"[^\w\s]", "", text)

```

```

    return text.strip().split()
def calculate_rms(audio_chunk):
    """
    计算音频片段 RMS 能量，用于简单 VAD
    """
    if len(audio_chunk) == 0:
        return 0
    return np.sqrt(np.mean(audio_chunk ** 2))
def create_recognizer(model_size):
    """
    创建 Faster-Whisper 识别器实例
    说明：
    - CTranslate2 底层为 C++ 实现
    - 推理阶段会释放 Python GIL
    - 因此 Python 多线程可以真正并发调用 GPU 推理
    """
    try:
        model = WhisperModel(
            model_size,
            device="cuda",
            compute_type="float16"
        )
        return model
    except Exception as e:
        print(f"模型加载失败: {e}")
        sys.exit(1)
# ===== 工作线程逻辑 =====
def worker_thread(thread_id, task_queue, recognizer):
    """
    工作线程函数：
    - 不断从任务队列中取出文件
    - 对单个音频文件进行处理
    """
    while True:
        try:
            # 非阻塞获取任务，队列空则退出线程
            task = task_queue.get_nowait()
        except queue.Empty:
            break
        file_id, audio_path, ref_text = task
        # 处理单个文件
        process_single_file(
            thread_id,
            file_id,
            audio_path,
            ref_text,
            recognizer
        )
        # 标记该任务已完成
        task_queue.task_done()

```

```

def process_single_file(thread_id, file_id, audio_path, ref_text, recognizer):
    """
    单文件处理逻辑：
    - 分片推理
    - 累计推理耗时
    - 计算文件级 RTF
    - 汇总 WER 统计
    """

    global g_total_proc_time, g_total_audio_dur
    global g_total_ref_words, g_total_distance
    # ===== 读取音频 =====
    try:
        audio, sample_rate = sf.read(audio_path, dtype="float32")
    except Exception:
        return
    file_audio_len = len(audio) / sample_rate
    if file_audio_len < 0.05:
        return
    # ===== 局部变量 =====
    chunk_size = int(sample_rate * CHUNK_DURATION)
    hyp_segments_list = []
    last_text = ""
    file_proc_time = 0.0
    # ===== 核心分片推理循环 =====
    for i in range(0, len(audio), chunk_size):
        chunk = audio[i: i + chunk_size]
        if len(chunk) < 160:
            continue
        # 单分片计时（用于 RTF）
        t_start = time.time()
        # 1. 简单能量 VAD
        rms = calculate_rms(chunk)
        if rms >= SILENCE_THRESHOLD:
            try:
                # 2. 模型推理（GPU 并发竞争点）
                segments, info = recognizer.transcribe(
                    chunk,
                    beam_size=1,
                    language="en",
                    vad_filter=True,
                    condition_on_previous_text=False,
                    temperature=0.0,
                    compression_ratio_threshold=2.0,
                    no_speech_threshold=0.4
                )
                # 3. 后处理（黑名单 + 去重）
                current_chunk_text = ""
                for segment in segments:
                    txt = segment.text.strip()
                    hallucinations = [

```

```

        "see you", "watching",
        "subtitles", "amara",
        "org", "bye"
    ]
    if any(h in txt.lower() for h in hallucinations):
        continue
    current_chunk_text += " " + txt
    current_chunk_text = current_chunk_text.strip()
    # 字数熔断 + 防重复
    if len(current_chunk_text.split()) <= MAX_WORDS_PER_SEC:
        if current_chunk_text and current_chunk_text != last_text:
            hyp_segments_list.append(current_chunk_text)
            last_text = current_chunk_text
    except Exception:
        pass
    # 分片计时结束
    t_end = time.time()
    file_proc_time += (t_end - t_start)
# ===== 单文件统计 =====
# 文件级实时率 (RTF)
curr_rtf = (
    file_proc_time / file_audio_len
    if file_audio_len > 0 else 0
)
# ===== 简化版 WER 计算 (仅用于全局汇总) =====
dist = 0
ref_len = 0
hyp_text_raw = " ".join(hyp_segments_list)
ref_words = normalize_text(ref_text)
if len(ref_words) >= MIN_WORD_COUNT:
    try:
        hyp_words = normalize_text(hyp_text_raw)
        out = jiwer.process_words(
            " ".join(ref_words),
            " ".join(hyp_words)
        )
        dist = (
            out.substitutions +
            out.deletions +
            out.insertions
        )
        ref_len = len(out.references[0])
    except:
        pass
# ===== 更新全局统计 (需加锁) =====
with stats_lock:
    g_total_proc_time += file_proc_time
    g_total_audio_dur += file_audio_len
    g_total_distance += dist
    g_total_ref_words += ref_len

```

```

# ===== 打印单文件结果（需加锁） =====
with print_lock:
    print(
        f"{file_id:<25} | "
        f"{file_proc_time:<12.4f} | "
        f"{file_audio_len:<10.2f} | "
        f"{curr_rtf:<8.4f}"
    )
# ===== 主程序 =====
def main():
    print("-" * 65)
    print(f"模式：并行处理 (Threads={PARALLEL_NUM})")
    print(f"正在加载模型 {MODEL_SIZE} ...")
    # 创建全局共享模型实例
    recognizer = create_recognizer(MODEL_SIZE)
    print("模型加载完成。")
    print("-" * 65)
    # 表头
    print(f"{'File ID':<25} | {'ProcTime(s)':<12} | {'Audio(s)':<10} | {'RTF':<8}")
    print("-" * 65)
    if not os.path.exists(LIBRISPEECH_DIR):
        print(f"错误：目录不存在 {LIBRISPEECH_DIR}")
        return
    # ===== 构建任务队列 =====
    task_queue = queue.Queue()
    file_count = 0
    print("正在扫描数据集并填充任务队列...")
    for root, dirs, files in os.walk(LIBRISPEECH_DIR):
        trans_files = [f for f in files if f.endswith(".trans.txt")]
        for trans_file in trans_files:
            trans_path = os.path.join(root, trans_file)
            ref_map = {}
            try:
                with open(trans_path, "r", encoding="utf-8") as f:
                    for line in f:
                        parts = line.strip().split(" ", 1)
                        if len(parts) == 2:
                            ref_map[parts[0]] = parts[1]
            except:
                continue
        for fid, txt in ref_map.items():
            audio_path = os.path.join(root, f"{fid}.wav")
            if not os.path.exists(audio_path):
                flac_path = os.path.join(root, f"{fid}.flac")
                if os.path.exists(flac_path):
                    audio_path = flac_path
                else:
                    continue
            # 入队任务
            task_queue.put((fid, audio_path, txt))

```

```

        file_count += 1
    print(
        f"任务队列填充完成, 共 {file_count} 个文件。"
        f"启动 {PARALLEL_NUM} 个线程并发处理..."
    )
    print("-" * 65)
    # ===== 启动线程池 =====
    threads = []
    wall_start_time = time.time()
    for i in range(PARALLEL_NUM):
        t = threading.Thread(
            target=worker_thread,
            args=(i, task_queue, recognizer)
        )
        t.daemon = True
        t.start()
        threads.append(t)
    # 等待所有任务完成
    task_queue.join()
    wall_end_time = time.time()
    # ===== 汇总统计 =====
    total_wall_time = wall_end_time - wall_start_time
    print("-" * 65)
    print("并发评测完成统计:")
    print(f"并行线程数: {PARALLEL_NUM}")
    print(f"总音频时长: {g_total_audio_dur:.2f} 秒")
    print(f"累积处理耗时: {g_total_proc_time:.2f} 秒 (各线程推理时间之和)")
    print(f"墙钟耗时: {total_wall_time:.2f} 秒 (用户实际等待时间)")
    if g_total_audio_dur > 0:
        # A: 平均单路推理实时率 (反映资源竞争)
        avg_latency_rtf = g_total_proc_time / g_total_audio_dur
        # B: 系统吞吐实时率 (反映并发整体效率)
        effective_rtf = total_wall_time / g_total_audio_dur
        print("-" * 30)
        print(f"★ 平均单路 RTF: {avg_latency_rtf:.5f}")
        print(f"★ 系统吞吐 RTF: {effective_rtf:.5f}")
        print(f"系统吞吐倍速: {1 / effective_rtf:.2f}x")
    if g_total_ref_words > 0:
        print(f"平均 WER: {g_total_distance / g_total_ref_words:.2%}")
    print("=" * 65)
if __name__ == "__main__":
    main()

```

```

=====
文件名称: Faster-Whisper_LibriSpeech_Parallel_TTFT.py
=====

```

```

import os
import sys
import re

```



```

import time
import soundfile as sf
import numpy as np
import threading
import queue
from faster_whisper import WhisperModel
# ===== 配置区域 =====
# LibriSpeech 测试集路径（已转换为 wav/flac）
LIBRISPEECH_DIR = "/opt/Audio_Datasets/LibriSpeech_WAV/test-clean"
# Faster-Whisper 模型规格
MODEL_SIZE = "medium.en"
# 滑动窗口分片长度（秒）
# 表示每 CHUNK_DURATION 秒送一次音频给模型
CHUNK_DURATION = 1.0 # 1 秒分片
# 并发线程数量
# 多线程共享同一个 GPU 模型实例
# 并发数越高，GPU 排队越明显，TTFT 会增大
PARALLEL_NUM = 1
# ===== VAD & 过滤参数 =====
# RMS 能量阈值，用于简单能量型 VAD
# 低于该值的音频块直接丢弃，不送入模型
SILENCE_THRESHOLD = 0.01
# 单秒内允许的最大单词数
# 用于过滤明显不合理的“幻觉文本”
MAX_WORDS_PER_SEC = 6
# ===== 全局共享资源 =====
# 打印锁，防止多线程输出交叉
print_lock = threading.Lock()
# 统计数据锁
stats_lock = threading.Lock()
# 存储每个音频文件的 TTFT 结果
g_ttft_list = []
# 已处理文件数量（保留字段）
g_processed_count = 0
# ===== 辅助函数 =====
def calculate_rms(audio_chunk):
    """
    计算音频分片的 RMS（Root Mean Square）能量
    用于快速判断是否存在有效语音
    """
    if len(audio_chunk) == 0:
        return 0
    return np.sqrt(np.mean(audio_chunk ** 2))
def create_recognizer(model_size):
    """
    创建 Faster-Whisper 识别器
    - 使用 CUDA 加速
    - 使用 float16 降低显存占用并提升吞吐
    """
    try:

```

```

        print(f"正在加载 GPU 模型 (float16)...")
        model = WhisperModel(
            model_size,
            device="cuda",
            compute_type="float16"
        )
        return model
    except Exception as e:
        print(f"模型加载失败: {e}")
        sys.exit(1)

# ===== 工作线程逻辑 =====
def worker_thread(thread_id, task_queue, recognizer):
    """
    工作线程函数：
    - 从任务队列中不断取音频文件
    - 对每个文件计算 TTFT
    """
    while True:
        try:
            # 非阻塞方式获取任务
            task = task_queue.get_nowait()
        except queue.Empty:
            # 队列为空，线程退出
            break
        file_id, audio_path = task
        # 核心逻辑：计算该音频的 TTFT
        ttft = process_until_first_token(file_id, audio_path, recognizer)
        # 保存统计结果
        if ttft is not None:
            with stats_lock:
                g_ttft_list.append(ttft)
        task_queue.task_done()

def process_until_first_token(file_id, audio_path, recognizer):
    """
    核心函数：
    采用“滑动窗口 + 早停”策略，
    在检测到第一个有效文本输出后立即停止处理。
    返回：
        TTFT (Time To First Token)，单位：秒
    """
    try:
        # 读取音频为 float32
        audio, sample_rate = sf.read(audio_path, dtype="float32")
    except Exception:
        return None
    # 忽略极短音频
    if len(audio) < int(sample_rate * 0.05):
        return None
    # 每个分片对应的采样点数量
    chunk_size = int(sample_rate * CHUNK_DURATION)

```

```

# ===== 滑动窗口推理 =====
# 按固定步长（1 秒）向前滑动
for i in range(0, len(audio), chunk_size):
    chunk = audio[i: i + chunk_size]
    # 丢弃过短的分片
    if len(chunk) < 160:
        continue
    # === 关键指标 1: 音频等待时间 ===
    # 第 i 个分片的“真实时间位置”
    # 例如:
    #   第 0 秒分片 → 等待 1 秒
    #   第 1 秒分片 → 等待 2 秒
    audio_wait_time = (i / sample_rate) + CHUNK_DURATION
    # === 开始计时: 模型计算延迟 ===
    t_start = time.time()
    # 1. 能量型 VAD 预判
    rms = calculate_rms(chunk)
    valid_text_found = False
    first_text = ""
    # 只有超过能量阈值的分片才送入 GPU
    if rms >= SILENCE_THRESHOLD:
        try:
            # 2. GPU 推理（并发竞争点）
            segments, info = recognizer.transcribe(
                chunk,
                beam_size=1,                    # 首字延迟场景不需要 beam search
                language="en",
                vad_filter=True,                # 启用模型内部 VAD
                condition_on_previous_text=False, # 分片间不传递上下文
                temperature=0.0,
                no_speech_threshold=0.4
            )
            current_chunk_text = ""
            # 3. 文本过滤（去除常见 Whisper 幻觉）
            for segment in segments:
                txt = segment.text.strip()
                hallucinations = [
                    "see you", "watching",
                    "subtitles", "amara", "org", "bye"
                ]
                if any(h in txt.lower() for h in hallucinations):
                    continue
                current_chunk_text += " " + txt
            current_chunk_text = current_chunk_text.strip()
            # 4. 判断是否为“有效首字”
            if (
                current_chunk_text and
                len(current_chunk_text.split()) <= MAX_WORDS_PER_SEC
            ):
                valid_text_found = True

```

```

        first_text = current_chunk_text
    except Exception:
        # 推理失败直接跳过该分片
        pass
    # === 结束计时 ===
    t_end = time.time()
    # 模型计算耗时 (含 GPU 排队)
    calc_duration = t_end - t_start
    # ===== TTFT 计算 =====
    if valid_text_found:
        # TTFT = 音频等待时间 + 模型计算时间
        ttft = audio_wait_time + calc_duration
        with print_lock:
            print(
                f"File: {file_id:<20} | "
                f"TTFT: {ttft:.4f}s "
                f"(Wait:{audio_wait_time:.0f}s + Calc:{calc_duration:.4f}s) | "
                f"Text: {first_text[:20]}..."
            )
        # 【早停机制】
        # 一旦拿到首字, 立即停止后续分片处理
        return ttft
    # 整个音频未检测到有效语音
    with print_lock:
        print(f"File: {file_id:<20} | [No Speech Found]")
    return None
# ===== 主程序 =====
def main():
    print("-" * 65)
    print(f"模式: GPU 并发测试 (Threads={PARALLEL_NUM})")
    print(f"设备: CUDA (float16)")
    print(f"指标: 首字延迟 (TTFT)")
    print("-" * 65)
    # 初始化模型 (所有线程共享)
    recognizer = create_recognizer(MODEL_SIZE)
    print("模型加载完成。")
    print("-" * 65)
    task_queue = queue.Queue()
    file_count = 0
    if not os.path.exists(LIBRISPEECH_DIR):
        print(f"目录不存在: {LIBRISPEECH_DIR}")
        return
    # 扫描音频文件并加入任务队列
    print("正在扫描数据集...")
    for root, dirs, files in os.walk(LIBRISPEECH_DIR):
        for f in files:
            if f.endswith(".wav") or f.endswith(".flac"):
                file_id = os.path.splitext(f)[0]
                audio_path = os.path.join(root, f)
                task_queue.put((file_id, audio_path))

```

```

        file_count += 1
    print(f"共扫描到 {file_count} 个文件。启动 {PARALLEL_NUM} 个线程开始并发评测...")
    print(f"{' File ID':<20} | {' TTFT Result (Wait + Calc)':<40} | {'First Text'}")
    print("-" * 80)
    # 启动并发线程
    threads = []
    for i in range(PARALLEL_NUM):
        t = threading.Thread(
            target=worker_thread,
            args=(i, task_queue, recognizer)
        )
        t.daemon = True
        t.start()
        threads.append(t)
    # 等待所有任务完成
    task_queue.join()
    # ===== 结果汇总 =====
    print("-" * 80)
    print("GPU 并发 TTFT 评测完成")
    print(f"文件总数: {file_count}")
    print(f"有输出的文件数: {len(g_ttft_list)}")
    if len(g_ttft_list) > 0:
        avg_ttft = sum(g_ttft_list) / len(g_ttft_list)
        min_ttft = min(g_ttft_list)
        max_ttft = max(g_ttft_list)
        # 计算标准差, 用于衡量延迟抖动
        std_dev = np.std(g_ttft_list)
        print(f"★ 平均首字延迟 (Avg TTFT): {avg_ttft:.4f} 秒")
        print(f" - 最小 TTFT: {min_ttft:.4f} 秒")
        print(f" - 最大 TTFT: {max_ttft:.4f} 秒")
        print(f" - 延迟波动 (Std Dev): {std_dev:.4f} 秒")
        print("-" * 40)
        print("注意: 并发场景下 Calc 时间包含 GPU 排队等待时间。")
    else:
        print("未检测到有效语音。")
    print("-" * 80)
if __name__ == "__main__":
    main()

```

```

=====
文件名称: Faster-Whisper_LibriSpeech_RTF.py
=====

```

```

import os
import sys
import re
import time
import soundfile as sf
import numpy as np
from faster_whisper import WhisperModel

```

```

import jiwer
# ===== 配置区域 =====
# LibriSpeech 测试集路径（已预处理为 wav / flac）
LIBRISPEECH_DIR = "/opt/Audio_Datasets/LibriSpeech_WAV/test-clean"
# Faster-Whisper 使用的模型规格
MODEL_SIZE = "medium.en"
# 滑动窗口分片长度（秒）
# 表示每 CHUNK_DURATION 秒送一次音频进行推理
CHUNK_DURATION = 1.0 # 1 秒分片
# ===== VAD & 过滤参数 =====
# RMS 能量阈值，用于简单能量型 VAD
# 低于该值的音频分片直接跳过，不参与推理
SILENCE_THRESHOLD = 0.01
# 单秒允许的最大单词数
# 用于过滤明显异常或幻觉文本
MAX_WORDS_PER_SEC = 6
# 最少参考词数阈值
# 仅影响是否参与 WER 统计，不影响 RTF 统计
MIN_WORD_COUNT = 3
# ===== 辅助函数 =====
def normalize_text(text):
    """
    文本归一化函数：
    - 转小写
    - 去除标点符号
    - 按空格切分为单词序列
    用于 WER 计算，确保参考文本和识别文本规则一致
    """
    if not text:
        return []
    text = text.lower()
    text = re.sub(r"[^\w\s]", "", text)
    words = text.strip().split()
    return words
def calculate_rms(audio_chunk):
    """
    计算音频分片的 RMS（Root Mean Square）能量
    用于快速判断该分片是否包含有效语音
    """
    if len(audio_chunk) == 0:
        return 0
    return np.sqrt(np.mean(audio_chunk ** 2))
def create_recognizer(model_size):
    """
    初始化 Faster-Whisper 识别模型
    - 使用 CUDA 加速
    - 使用 float16 以提高吞吐、降低显存占用
    """
    try:
        model = WhisperModel(

```

```

    try:
        # ----- 2. 识别 -----
        hyp_text_raw = transcribe_wav_file(
            model,
            audio_path
        )
        if hyp_text_raw.startswith("Error"):
            print(f"Skip {file_id}: {hyp_text_raw}")
            continue
        # ----- 3. WER -----
        hyp_words_list = normalize_text(hyp_text_raw)
        ref_str_clean = " ".join(ref_words_list)
        hyp_str_clean = " ".join(hyp_words_list)
        out = jiwer.process_words(
            ref_str_clean,
            hyp_str_clean
        )
        dist = (
            out.substitutions +
            out.deletions +
            out.insertions
        )
        curr_len = (
            len(out.references[0])
            if out.references else 0
        )
        total_distance += dist
        total_ref_words += curr_len
        file_count += 1
        print(
            f"File: {file_id} | "
            f"WER: {out.wer:.4f} | "
            f"Dist: {dist} | "
            f"RefWords: {curr_len}"
        )
    except Exception as e:
        print(f"Error {file_id}: {e}")
        continue
except Exception as e:
    print(f"读取 text 文件失败: {e}")
    return

# ===== 汇总 =====
print("\n" + "=" * 50)
print("评测完成")
print(f"有效处理文件数: {file_count}")
print(f"跳过文件数: {skipped_count}")
if total_ref_words > 0:
    avg_wer = total_distance / total_ref_words
    print(f"总编辑距离: {total_distance}")
    print(f"总参考单词数: {total_ref_words}")

```

```

        print("-" * 25)
        print(f"平均错词率 (Average WER): {avg_wer:.2%}")
        print("=" * 50)
if __name__ == "__main__":
    main()

```

```

=====
文件名称: Faster-Whisper_TEDLIUM_WER.py
=====

```

```

import os
import sys
import re
import soundfile as sf
import numpy as np
from faster_whisper import WhisperModel
import jiwer
# ===== 配置区域 =====
# 数据集根目录
DATASET_ROOT = "/opt/Audio_Datasets/TEDLIUM_WAV"
# 文本文件路径
TRANS_FILE = os.path.join(DATASET_ROOT, "text.txt")
MODEL_SIZE = "medium.en"
CHUNK_DURATION = 1.0 # 1S 分片
# 静音阈值: RMS 能量低于此值视为静音
SILENCE_THRESHOLD = 0.01
# 最大词数限制: 1 秒内人类很难说超过 6 个单词
MAX_WORDS_PER_SEC = 6
# 过滤条件: 仅评测有效单词数 >= 3 的样本
MIN_WORD_COUNT = 3
# ===== 辅助函数 =====
def normalize_text(text):
    """
    基础文本标准化:
    1. 转小写
    2. 去除标点 (保留字母、数字、空格)
    3. 分词
    """
    if not text:
        return []
    text = text.lower()
    text = re.sub(r"[^\w\s]", "", text) # 去除标点
    words = text.strip().split()
    return words
def clean_tedlium_text(raw_text):
    """
    TEDLIUM 专用清洗:
    1. 去除 <COMMA>, <SIL>, <OTHER> 等所有尖括号标签
    2. 标准化处理
    3. 返回清洗后的单词列表
    """

```



```

"""
    if not raw_text:
        return []
    # 1. 使用正则去除 <...> 内容
    # 替换为空格防止粘连, 例如 "hello<SIL>world" -> "hello world"
    text_no_tags = re.sub(r'<[^>]+>', ' ', raw_text)
    # 2. 调用标准归一化进行分词
    words = normalize_text(text_no_tags)
    return words
def calculate_rms(audio_chunk):
    """计算音频切片的均方根能量"""
    if len(audio_chunk) == 0:
        return 0
    return np.sqrt(np.mean(audio_chunk ** 2))
def build_audio_file_map(root_dir):
    """
    递归遍历目录, 建立文件名(无后缀)到完整路径的映射
    解决音频文件分散在多个子目录的问题
    """
    print(f"正在扫描音频文件: {root_dir} ...")
    audio_map = {}
    count = 0
    for root, dirs, files in os.walk(root_dir):
        for file in files:
            if file.endswith(".wav"):
                # 假设文件名为 ID.wav, 取 ID 作为 key
                file_id = os.path.splitext(file)[0]
                full_path = os.path.join(root, file)
                audio_map[file_id] = full_path
                count += 1
    print(f"扫描完成, 找到 {count} 个 wav 文件。")
    return audio_map
# ===== Faster-Whisper 初始化 =====
def create_recognizer(model_size):
    try:
        model = WhisperModel(model_size, device="cuda", compute_type="float16")
        return model
    except Exception as e:
        print(f"模型加载失败: {e}")
        sys.exit(1)
# ===== 主程序 =====
def main():
    print("-" * 50)
    print(f"正在加载模型 {MODEL_SIZE} (TEDLIUM 适配版)...")
    recognizer = create_recognizer(MODEL_SIZE)
    print("模型加载完成。")
    print("-" * 50)
    # 检查基本路径
    if not os.path.isfile(TRANS_FILE):
        print(f"错误: 找不到文本文件: {TRANS_FILE}")

```

```

    return
if not os.path.isdir(DATASET_ROOT):
    print(f"错误：数据集目录不存在：{DATASET_ROOT}")
    return
# 1. 构建音频文件路径索引（处理子目录）
audio_path_map = build_audio_file_map(DATASET_ROOT)
# 2. 读取并过滤文本
print(f"正在读取并处理文本文件：{TRANS_FILE}")
valid_tasks = {} # {file_id: ref_word_list}
skipped_count = 0
missing_audio_count = 0
try:
    with open(TRANS_FILE, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            # 拆分 ID 和 文本（通常以第一个空格分隔）
            parts = line.split(maxsplit=1)
            # 可能存在空文本行，仅有 ID
            file_id = parts[0]
            raw_content = parts[1] if len(parts) > 1 else ""
            # 检查是否存在对应的音频文件
            if file_id not in audio_path_map:
                # 某些数据集 text 可能包含训练集数据，而音频目录只有测试集
                # 这种情况算作“非本集数据”，不算错误，但需要跳过
                missing_audio_count += 1
                continue
            # 清洗文本（去标签、去标点、转小写）
            ref_words = clean_tedlium_text(raw_content)
            # 过滤规则：单词数 >= 3
            if len(ref_words) >= MIN_WORD_COUNT:
                valid_tasks[file_id] = ref_words
            else:
                skipped_count += 1
except Exception as e:
    print(f"读取文本文件时出错：{e}")
    return
print(f"文本处理完成。")
print(f"匹配到音频且符合评测条件的样本数：{len(valid_tasks)}")
print(f"因文本过短或仅含标签跳过：{skipped_count}")
if missing_audio_count > 0:
    print(f"在 wav 目录中未找到对应音频的条目数：{missing_audio_count}")
print("--" * 50)
# 3. 开始遍历评测
total_distance = 0
total_ref_words = 0
processed_count = 0
# 对 ID 进行排序，保证输出顺序一致
sorted_file_ids = sorted(valid_tasks.keys())

```

```

for file_id in sorted_file_ids:
    ref_words_list = valid_tasks[file_id]
    audio_path = audio_path_map[file_id]
    try:
        # 读取音频
        audio, sample_rate = sf.read(audio_path, dtype="float32")
    except Exception as e:
        print(f"读取音频失败 {audio_path}: {e}")
        continue
    # 忽略极短音频 (小于 0.05s)
    if len(audio) < int(sample_rate * 0.05):
        continue
    # ===== 核心推理逻辑 (切片 + VAD + 熔断) =====
    chunk_size = int(sample_rate * CHUNK_DURATION)
    hyp_segments_list = []
    last_text = ""
    for i in range(0, len(audio), chunk_size):
        chunk = audio[i: i + chunk_size]
        if len(chunk) < 160: continue
        # --- 1. 能量 VAD ---
        rms = calculate_rms(chunk)
        if rms < SILENCE_THRESHOLD:
            continue
        # --- 2. 推理 ---
        try:
            segments, info = recognizer.transcribe(
                chunk,
                beam_size=1,
                language="en",
                vad_filter=True,
                condition_on_previous_text=False,
                temperature=0.0,
                compression_ratio_threshold=2.0,
                no_speech_threshold=0.4
            )
            current_chunk_text = ""
            for segment in segments:
                txt = segment.text.strip()
                # --- 3. 黑名单过滤 ---
                hallucinations = ["see you", "watching", "subtitles", "amara", "org", "bye"]
                if any(h in txt.lower() for h in hallucinations):
                    continue
                current_chunk_text += " " + txt
            current_chunk_text = current_chunk_text.strip()
            # --- 4. 字数熔断 ---
            words_in_chunk = len(current_chunk_text.split())
            if words_in_chunk > MAX_WORDS_PER_SEC:
                continue
            # --- 5. 防重复 ---
            if current_chunk_text and current_chunk_text != last_text:

```

```

        hyp_segments_list.append(current_chunk_text)
        last_text = current_chunk_text
    except Exception as e:
        continue
    hyp_text_raw = " ".join(hyp_segments_list)
    # =====
    # ===== 评测计算 =====
    hyp_words_list = normalize_text(hyp_text_raw)
    ref_str_clean = " ".join(ref_words_list)
    hyp_str_clean = " ".join(hyp_words_list)
    try:
        out = jiwer.process_words(ref_str_clean, hyp_str_clean)
        curr_dist = out.substitutions + out.deletions + out.insertions
        curr_wer = out.wer
        curr_len = len(out.references[0]) if out.references else 0
        total_distance += curr_dist
        total_ref_words += curr_len
        processed_count += 1
        # 打印当前文件的 WER
        print(f"File: {file_id} | WER: {curr_wer:.4f} | Dist: {curr_dist} | RefWords: {curr_len}")
    except Exception as e:
        print(f"Jiwer 计算错误 {file_id}: {e}")
    # ===== 结果汇总 =====
    print("\n" + "=" * 50)
    print("评测完成 (TEDLIUM Test)")
    print(f"实际处理文件数: {processed_count}")
    if total_ref_words > 0:
        avg_wer = total_distance / total_ref_words
        print(f"总编辑距离: {total_distance}")
        print(f"总参考单词数: {total_ref_words}")
        print("-" * 25)
        print(f"平均错词率 (Average WER): {avg_wer:.2%}")
    else:
        print("未生成任何有效统计数据。")
    print("=" * 50)
if __name__ == "__main__":
    main()

=====
文件名称: Sherpa-onnx_AMICorpus_WER.py
=====

import os
import sys
import re
import soundfile as sf
import sherpa_onnx
import jiwer
# =====
# 配置区域

```

```

# =====
# AMICorpus 数据集根目录（包含 wav + 对应 txt 标注）
AMI_DIR = "/opt/Audio_Datasets/AMICorpus"
# Sherpa-ONNX 流式 Zipformer 模型目录
MODEL_DIR = "./sherpa-onnx-streaming-zipformer-en-2023-06-21"
# 流式输入的音频分片长度（秒）
# 0.1s  $\approx$  100ms，模拟真实在线语音输入
CHUNK_DURATION = 0.1
# 最大评测文件数（用于快速验证）
MAX_FILES = 10
# =====
# 1. 文本清洗函数
# =====
def normalize_ami_text(text):
    """
    AMICorpus 文本标准化函数：
    1. 替换回车 / 换行，保证文本连续
    2. 转为小写（大小写不计入 WER）
    3. 去除标点符号
    4. 返回空格分隔的单词字符串
    说明：
    - 该处理方式与 ASR 评测中常见 WER 计算保持一致
    - Ref 与 Hyp 使用完全相同的标准化规则
    """
    if not text:
        return ""
    # 去除换行符，避免影响分词
    text = text.replace("\n", " ").replace("\r", " ")
    # 统一转小写
    text = text.lower()
    # 去除除字母、数字、空格以外的字符
    text = re.sub(r"[^a-z0-9\s]", "", text)
    # 规范空格并返回
    words = text.strip().split()
    return " ".join(words)
# =====
# 2. Sherpa-ONNX 在线识别器初始化
# =====
def create_recognizer(model_dir):
    """
    创建 Sherpa-ONNX OnlineRecognizer (RNNT / Zipformer)
    功能：
    - 自动从模型目录中定位 encoder / decoder / joiner
    - 使用 Transducer 在线识别接口
    - 禁用端点检测，由程序手动控制流结束
    """
    tokens_path = os.path.join(model_dir, "tokens.txt")
    encoder_path = ""
    decoder_path = ""
    joiner_path = ""

```

```

# 检查模型目录
if not os.path.exists(model_dir):
    print(f"错误: 模型目录不存在 {model_dir}")
    sys.exit(1)
# 自动查找 onnx 模型文件
for f in os.listdir(model_dir):
    if f.startswith("encoder-") and f.endswith(".onnx"):
        encoder_path = os.path.join(model_dir, f)
    elif f.startswith("decoder-") and f.endswith(".onnx"):
        decoder_path = os.path.join(model_dir, f)
    elif f.startswith("joiner-") and f.endswith(".onnx"):
        joiner_path = os.path.join(model_dir, f)
# 校验模型文件完整性
if not (os.path.exists(tokens_path) and encoder_path and decoder_path and joiner_path):
    print(f"错误: 模型文件缺失。\\nTokens: {tokens_path}")
    sys.exit(1)
print(
    f"加载模型:\\n"
    f" Encoder: {os.path.basename(encoder_path)}\\n"
    f" Decoder: {os.path.basename(decoder_path)}\\n"
    f" Joiner : {os.path.basename(joiner_path)}"
)
# 初始化 OnlineRecognizer (流式 RNNT)
try:
    recognizer = sherpa_onnx.OnlineRecognizer.from_transducer(
        tokens=tokens_path,
        encoder=encoder_path,
        decoder=decoder_path,
        joiner=joiner_path,
        num_threads=1,
        sample_rate=16000,
        feature_dim=80,
        decoding_method="greedy_search",
        enable_endpoint_detection=False, # 不自动截断
        provider="cpu"
    )
except Exception as e:
    # 某些版本 sherpa-onnx 不支持 endpoint 参数
    print(f"初始化警告: {e}, 尝试默认参数...")
    recognizer = sherpa_onnx.OnlineRecognizer.from_transducer(
        tokens=tokens_path,
        encoder=encoder_path,
        decoder=decoder_path,
        joiner=joiner_path,
        num_threads=1,
        sample_rate=16000,
        feature_dim=80,
        decoding_method="greedy_search",
        provider="cpu"
    )

```

```

    return recognizer
# =====
# 3. 主程序
# =====
def main():
    print("-" * 50)
    print(f"初始化 Sherpa-ONNX (AMICorpus 流式评测, Limit={MAX_FILES})...")
    recognizer = create_recognizer(MODEL_DIR)
    print("模型加载完成。")
    print("-" * 50)
    # 数据集路径检查
    if not os.path.exists(AMI_DIR):
        print(f"错误: 数据集目录不存在 {AMI_DIR}")
        return
    # 评测统计量
    total_distance = 0      # 总编辑距离
    total_ref_words = 0     # 总参考单词数
    processed_count = 0     # 成功处理的文件数
    stop_processing = False # 达到 MAX_FILES 后停止
    print(f"开始遍历数据集: {AMI_DIR}")
    # 递归遍历 AMICorpus
    for root, dirs, files in os.walk(AMI_DIR):
        if stop_processing:
            break
        wav_files = [f for f in files if f.endswith(".wav")]
        for wav_filename in wav_files:
            if stop_processing:
                break
            wav_path = os.path.join(root, wav_filename)
            txt_filename = os.path.splitext(wav_filename)[0] + ".txt"
            txt_path = os.path.join(root, txt_filename)
            # 跳过无标注文件
            if not os.path.exists(txt_path):
                continue
            # ----- 1. 读取并清洗参考文本 -----
            try:
                with open(txt_path, "r", encoding="utf-8") as f:
                    raw_text = f.read()
            except Exception as e:
                print(f"读取文本失败 {txt_path}: {e}")
                continue
            ref_clean = normalize_ami_text(raw_text)
            ref_words_list = ref_clean.split()
            # 过滤过短语音
            if len(ref_words_list) < 3:
                continue
            # ----- 2. 读取音频 -----
            try:
                audio, sample_rate = sf.read(wav_path, dtype="float32")
            except Exception as e:

```

```

        print(f"读取音频失败 {wav_path}: {e}")
        continue
    if sample_rate != 16000:
        print(f"跳过非 16k 音频: {wav_filename}")
        continue
    if len(audio) < int(sample_rate * 0.05):
        continue
    duration_sec = len(audio) / sample_rate
    # ----- 3. 流式推理（核心部分） -----
    # 创建在线流
    stream = recognizer.create_stream()
    # 滑动窗口分片大小
    chunk_size = int(sample_rate * CHUNK_DURATION)
    # 按 100ms 切片逐块送入模型
    for i in range(0, len(audio), chunk_size):
        chunk = audio[i: i + chunk_size]
        # 接收音频流
        stream.accept_waveform(sample_rate, chunk)
        # 当模型准备好时持续解码
        while recognizer.is_ready(stream):
            recognizer.decode_stream(stream)
    # 通知流结束
    stream.input_finished()
    # 解码残余帧
    while recognizer.is_ready(stream):
        recognizer.decode_stream(stream)
    # 获取最终识别结果
    result_obj = recognizer.get_result(stream)
    hyp_text_raw = (
        result_obj if isinstance(result_obj, str)
        else getattr(result_obj, 'text', str(result_obj))
    )
    # ----- 4. WER 评测 -----
    hyp_clean = normalize_ami_text(hyp_text_raw)
    try:
        out = jiwer.process_words(ref_clean, hyp_clean)
        curr_dist = out.substitutions + out.deletions + out.insertions
        curr_wer = out.wer
        curr_len = len(out.references[0]) if out.references else 0
        total_distance += curr_dist
        total_ref_words += curr_len
        processed_count += 1
        # 输出单文件评测信息
        print(f"[{processed_count}/{MAX_FILES}] File: {wav_filename}")
        print(
            f"    Duration: {duration_sec:.2f}s | "
            f"Words: {curr_len} | Dist: {curr_dist} | WER: {curr_wer:.4f}"
        )
    # 达到测试数量限制
    if processed_count >= MAX_FILES:

```



```

        stop_processing = True
    except Exception as e:
        print(f"评测计算错误 {wav_filename}: {e}")

# =====
# 结果汇总
# =====
print("\n" + "=" * 50)
print("测试结束 (Limit Reached)")
print(f"处理文件总数: {processed_count}")
if total_ref_words > 0:
    avg_wer = total_distance / total_ref_words
    print(f"总编辑距离: {total_distance}")
    print(f"总参考单词数: {total_ref_words}")
    print("-" * 25)
    print(f"平均错词率 (Average WER): {avg_wer:.2%}")
else:
    print("未生成任何有效统计数据。")
print("=" * 50)
if __name__ == "__main__":
    main()

```

```

=====
文件名称: Sherpa-onnx_LibriSpeech_WER.py
=====

```

```

import os
import sys
import re
import soundfile as sf
import sherpa_onnx
import jiwer
# ===== 配置区域 =====
# LibriSpeech (已转为 WAV) 的测试集目录
LIBRISPEECH_DIR = "/opt/Audio_Datasets/LibriSpeech_WAV/test-clean"
# sherpa-onnx 流式 Zipformer 英文模型目录
MODEL_DIR = "./sherpa-onnx-streaming-zipformer-en-2023-06-21"
# 流式推理时的音频分块长度 (秒)
# 0.1s = 100ms, 对应模拟真实流式输入
CHUNK_DURATION = 0.1
# ===== 文本标准化函数 =====
def normalize_text(text):
    """
    对参考文本和识别结果做统一的文本标准化处理, 用于 WER 计算:
    1. 全部转为小写
    2. 去除标点符号 (仅保留字母、数字和空格)
    3. 按空格切分为单词列表
    """
    if not text:
        return []
    # 1. 转小写

```

```

    text = text.lower()
    # 2. 正则去除标点符号
    # \w 匹配字母、数字和下划线
    # [^\w\s] 表示“非字母数字 且 非空格”的字符
    text = re.sub(r"[^\w\s]", "", text)
    # 3. 去除首尾空格并按空格分词
    words = text.strip().split()
    return words
# ===== Sherpa-onnx 初始化 =====
def create_recognizer(model_dir):
    """
    根据模型目录动态加载 encoder / decoder / joiner,
    并初始化 sherpa-onnx 的 OnlineRecognizer (流式识别器)
    """
    tokens_path = os.path.join(model_dir, "tokens.txt")
    encoder_path = ""
    decoder_path = ""
    joiner_path = ""
    # 检查模型目录是否存在
    if not os.path.exists(model_dir):
        print(f"错误: 模型目录不存在 {model_dir}")
        sys.exit(1)
    # 自动识别模型文件 (避免硬编码文件名)
    for f in os.listdir(model_dir):
        if f.startswith("encoder-") and f.endswith(".onnx"):
            encoder_path = os.path.join(model_dir, f)
        elif f.startswith("decoder-") and f.endswith(".onnx"):
            decoder_path = os.path.join(model_dir, f)
        elif f.startswith("joiner-") and f.endswith(".onnx"):
            joiner_path = os.path.join(model_dir, f)
    # 检查模型文件是否齐全
    if not (os.path.exists(tokens_path) and encoder_path and decoder_path and joiner_path):
        print(f"错误: 模型文件缺失。\\nTokens: {tokens_path}")
        sys.exit(1)
    print(
        f"加载模型:\\n"
        f"  Enc: {os.path.basename(encoder_path)}\\n"
        f"  Dec: {os.path.basename(decoder_path)}\\n"
        f"  Join: {os.path.basename(joiner_path)}"
    )
    # 初始化 OnlineRecognizer (Transducer 模型)
    try:
        recognizer = sherpa_onnx.OnlineRecognizer.from_transducer(
            tokens=tokens_path,
            encoder=encoder_path,
            decoder=decoder_path,
            joiner=joiner_path,
            num_threads=1,
            sample_rate=16000,
            feature_dim=80,

```

```

        decoding_method="greedy_search",
        enable_endpoint_detection=False, # 关闭端点检测, 避免短音频提前结束
        provider="cpu"
    )
except Exception as e:
    # 部分版本不支持 enable_endpoint_detection 参数, 做兼容处理
    print(f"初始化警告: {e}, 尝试默认参数...")
    recognizer = sherpa_onnx.OnlineRecognizer.from_transducer(
        tokens=tokens_path,
        encoder=encoder_path,
        decoder=decoder_path,
        joiner=joiner_path,
        num_threads=1,
        sample_rate=16000,
        feature_dim=80,
        decoding_method="greedy_search",
        provider="cpu"
    )
    return recognizer
# ===== 主程序 =====
def main():
    print("-" * 50)
    print("正在初始化 Sherpa-onnx...")
    recognizer = create_recognizer(MODEL_DIR)
    print("模型加载完成。")
    print("-" * 50)
    # 用于全局 WER 统计的累加变量
    total_distance = 0 # 总编辑距离 (S + D + I)
    total_ref_words = 0 # 总参考单词数
    file_count = 0 # 成功评测的音频文件数
    if not os.path.exists(LIBRISPEECH_DIR):
        print(f"错误: 数据集目录不存在 {LIBRISPEECH_DIR}")
        return
    print(f"开始遍历数据集: {LIBRISPEECH_DIR}")
    # 遍历 LibriSpeech 目录结构
    for root, dirs, files in os.walk(LIBRISPEECH_DIR):
        # 查找对应的转写文件
        trans_files = [f for f in files if f.endswith(".trans.txt")]
        for trans_file in trans_files:
            trans_path = os.path.join(root, trans_file)
            ref_map = {}
            # 读取 *.trans.txt, 构建 file_id -> 参考文本 的映射
            try:
                with open(trans_path, "r", encoding="utf-8") as f:
                    for line in f:
                        parts = line.strip().split(" ", 1)
                        if len(parts) == 2:
                            file_id, text = parts
                            ref_map[file_id] = text
            except Exception as e:

```

```

        print(f"读取文本文件失败 {trans_path}: {e}")
        continue
# 遍历该目录下的每一条语音
for file_id, ref_text in ref_map.items():
    audio_filename = f"{file_id}.wav"
    audio_path = os.path.join(root, audio_filename)
    # 兼容 flac 格式
    if not os.path.exists(audio_path):
        flac_path = os.path.join(root, f"{file_id}.flac")
        if os.path.exists(flac_path):
            audio_path = flac_path
        else:
            continue
    # 读取音频
    try:
        audio, sample_rate = sf.read(audio_path, dtype="float32")
    except Exception as e:
        print(f"读取音频失败 {audio_path}: {e}")
        continue
    # 只处理 16kHz 音频
    if sample_rate != 16000:
        print(f"警告: 采样率非 16k ({sample_rate}Hz) - 跳过 {audio_filename}")
        continue
    # 跳过极短音频（避免无意义统计）
    if len(audio) < int(sample_rate * 0.05):
        continue
    # ===== 核心推理逻辑 =====
    # 创建一个新的流（stream），对应一条音频
    stream = recognizer.create_stream()
    # 根据 CHUNK_DURATION 计算每个滑动窗口的采样点数
    # 0.1s * 16000Hz = 1600 samples
    chunk_size = int(sample_rate * CHUNK_DURATION)
    # 将整段音频按固定窗口分块送入模型
    # 这是典型的“滑动窗口 + 流式解码”处理方式
    for i in range(0, len(audio), chunk_size):
        chunk = audio[i: i + chunk_size]
        stream.accept_waveform(sample_rate, chunk)
        # 当模型内部积累了足够特征帧时进行解码
        while recognizer.is_ready(stream):
            recognizer.decode_stream(stream)
    # 通知模型音频输入结束
    stream.input_finished()
    # 解码剩余未处理的尾部帧
    while recognizer.is_ready(stream):
        recognizer.decode_stream(stream)
    # 获取识别结果
    result_obj = recognizer.get_result(stream)
    hyp_text_raw = (
        result_obj
        if isinstance(result_obj, str)

```

```

        else getattr(result_obj, 'text', str(result_obj))
    )
    # =====
    # ===== WER 评测逻辑 =====
    # 对参考文本和识别结果进行统一标准化
    ref_words_list = normalize_text(ref_text)
    hyp_words_list = normalize_text(hyp_text_raw)
    # jiwer 对 list 的语义是“多句子 batch”，
    # 因此需要将单词列表重新拼接成字符串
    ref_str_clean = " ".join(ref_words_list)
    hyp_str_clean = " ".join(hyp_words_list)
    try:
        # 计算 WER 及各类编辑距离
        out = jiwer.process_words(ref_str_clean, hyp_str_clean)
        curr_dist = (
            out.substitutions +
            out.deletions +
            out.insertions
        )
        curr_wer = out.wer
        # 当前语音的参考单词数
        curr_len = len(out.references[0]) if out.references else 0
        # 累加全局统计量
        total_distance += curr_dist
        total_ref_words += curr_len
        file_count += 1
        print(
            f"File: {os.path.basename(audio_path)} | "
            f"WER: {curr_wer:.4f} | "
            f"Dist: {curr_dist} | "
            f"RefWords: {curr_len}"
        )
    except Exception as e:
        print(f"Jiwer 计算错误 {audio_filename}: {e}")
        if len(ref_words_list) == 0:
            print(f"Ref 为空, Hyp 长度: {len(hyp_words_list)}")
    # ===== 结果汇总 =====
    print("\n" + "=" * 50)
    print("评测完成")
    print(f"处理文件总数: {file_count}")
    if total_ref_words > 0:
        avg_wer = total_distance / total_ref_words
        print(f"总编辑距离: {total_distance}")
        print(f"总参考单词数: {total_ref_words}")
        print("-" * 25)
        print(f"平均错词率 (Average WER): {avg_wer:.2%}")
    else:
        print("未处理任何有效文件。")
    print("=" * 50)
if __name__ == "__main__":

```

```
main()
```

```
=====
文件名称: Sherpa-onnx_TEDLIUM_WER.py
=====
```

```
import os
import sys
import re
import soundfile as sf
import sherpa_onnx
import jiwer

# ===== 配置区域 =====
# TEDLIUM 数据集根目录
# 典型结构:
#   TEDLIUM_WAV/
#   |----- text.txt          # 全部语音的转写索引
#   |----- legacy/
#   |       |----- train/sph/...  # 音频文件, 可能存在多层子目录
TEDLIUM_DIR = "/opt/Audio_Datasets/TEDLIUM_WAV"
# sherpa-onnx 流式 Zipformer 英文模型目录
MODEL_DIR = "./sherpa-onnx-streaming-zipformer-en-2023-06-21"
# 流式识别的音频分块时长(秒)
# 0.1s = 100ms, 用于模拟实时语音输入
CHUNK_DURATION = 0.1
# ===== 1. TEDLIUM 文本清洗函数 =====
def normalize_tedlium_text(text):
    """
    针对 TEDLIUM 数据集转写文本的专用标准化函数:
    1. 去除 TEDLIUM 特有的尖括号标签, 如 <COMMA>、<SIL>、<OTHER>
    2. 转为小写
    3. 去除标点和特殊符号, 仅保留字母、数字和空格
    4. 返回规范化后的单句字符串, 用于 WER 计算
    """
    if not text:
        return ""
    # 1. 去除尖括号标注内容
    # <COMMA>, <SIL>, <NOISE> 等
    text = re.sub(r"<[>]+>", " ", text)
    # 2. 转小写, 统一大小写
    text = text.lower()
    # 3. 去除非字母、非数字字符
    text = re.sub(r"[^a-z0-9\s]", "", text)
    # 4. 规范化空格(去除多余空格)
    words = text.strip().split()
    return " ".join(words)
# ===== 2. 建立音频文件索引 =====
def index_audio_files(root_dir):
    """
    递归遍历 TEDLIUM 数据集目录,
```

建立 {file\_id : 音频完整路径} 的映射表。

目的：

- 避免每次评测时重复遍历目录
- 解决 TEDLIUM 音频分散在多级子目录的问题

"""

```
audio_map = {}
print(f"正在建立音频文件索引: {root_dir} ...")
count = 0
for root, dirs, files in os.walk(root_dir):
    for f in files:
        # TEDLIUM 中音频可能为 wav 或 flac
        if f.endswith(".wav") or f.endswith(".flac"):
            # 使用不带后缀的文件名作为唯一 ID
            file_id = os.path.splitext(f)[0]
            full_path = os.path.join(root, f)
            audio_map[file_id] = full_path
            count += 1
print(f"索引建立完成, 共找到 {count} 个音频文件。")
return audio_map
```

# ===== 3. Sherpa-onnx 初始化 =====

def create\_recognizer(model\_dir):

"""

自动加载 Zipformer Transducer 模型,  
并初始化 sherpa-onnx 的 OnlineRecognizer (流式识别器)

"""

```
tokens_path = os.path.join(model_dir, "tokens.txt")
encoder_path = ""
decoder_path = ""
joiner_path = ""
```

# 模型目录存在性检查

```
if not os.path.exists(model_dir):
    print(f"错误: 模型目录不存在 {model_dir}")
    sys.exit(1)
```

# 自动匹配 encoder / decoder / joiner 文件

```
for f in os.listdir(model_dir):
    if f.startswith("encoder-") and f.endswith(".onnx"):
        encoder_path = os.path.join(model_dir, f)
    elif f.startswith("decoder-") and f.endswith(".onnx"):
        decoder_path = os.path.join(model_dir, f)
    elif f.startswith("joiner-") and f.endswith(".onnx"):
        joiner_path = os.path.join(model_dir, f)
```

# 检查模型文件完整性

```
if not (os.path.exists(tokens_path) and encoder_path and decoder_path and joiner_path):
    print(f"错误: 模型文件缺失。 \nTokens: {tokens_path}")
    sys.exit(1)
```

```
print(
    f"加载模型:\n"
    f"  Enc: {os.path.basename(encoder_path)}\n"
    f"  Dec: {os.path.basename(decoder_path)}\n"
    f"  Join: {os.path.basename(joiner_path)}"
```

```

    )
    try:
        # 初始化流式 Transducer 识别器
        recognizer = sherpa_onnx.OnlineRecognizer.from_transducer(
            tokens=tokens_path,
            encoder=encoder_path,
            decoder=decoder_path,
            joiner=joiner_path,
            num_threads=1,
            sample_rate=16000,
            feature_dim=80,
            decoding_method="greedy_search",
            enable_endpoint_detection=False, # 必须关闭, 避免短音频提前结束
            provider="cpu"
        )
    except Exception as e:
        # 兼容旧版本 sherpa-onnx
        print(f"初始化警告: {e}, 尝试默认参数...")
        recognizer = sherpa_onnx.OnlineRecognizer.from_transducer(
            tokens=tokens_path,
            encoder=encoder_path,
            decoder=decoder_path,
            joiner=joiner_path,
            num_threads=1,
            sample_rate=16000,
            feature_dim=80,
            decoding_method="greedy_search",
            provider="cpu"
        )
    return recognizer

# ===== 4. 主程序 =====
def main():
    print("-" * 50)
    print("初始化 Sherpa-onnx (TEDLIUM 评测)...")
    recognizer = create_recognizer(MODEL_DIR)
    print("模型加载完成。")
    print("-" * 50)
    # ----- Step 1: 建立音频索引 -----
    if not os.path.exists(TEDLIUM_DIR):
        print(f"错误: 数据集目录不存在 {TEDLIUM_DIR}")
        return
    audio_map = index_audio_files(TEDLIUM_DIR)
    # ----- Step 2: 读取文本索引 -----
    text_file_path = os.path.join(TEDLIUM_DIR, "text.txt")
    if not os.path.exists(text_file_path):
        print(f"错误: 找不到文本索引文件 {text_file_path}")
        return
    # 全局统计变量
    total_distance = 0          # 总编辑距离
    total_ref_words = 0         # 总参考单词数

```



```

file_count = 0          # 有效评测文件数
ignored_short_count = 0 # 过短样本数
missing_audio_count = 0 # 缺失音频文件数
print(f"开始评测: {text_file_path}")
# ----- Step 3: 遍历每条语音 -----
with open(text_file_path, "r", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if not line:
            continue
        # 每行格式: <file_id> <transcription>
        parts = line.split(maxsplit=1)
        if len(parts) < 2:
            continue
        file_id = parts[0]
        raw_ref_text = parts[1]
        # --- 文本清洗 ---
        ref_clean = normalize_tedlium_text(raw_ref_text)
        ref_words_list = ref_clean.split()
        # 过滤过短文本 (减少统计噪声)
        if len(ref_words_list) < 3:
            ignored_short_count += 1
            continue
        # --- 音频查找 ---
        if file_id not in audio_map:
            missing_audio_count += 1
            continue
        audio_path = audio_map[file_id]
        # --- 音频读取 ---
        try:
            audio, sample_rate = sf.read(audio_path, dtype="float32")
        except Exception as e:
            print(f"读取失败 {audio_path}: {e}")
            continue
        if sample_rate != 16000:
            print(f"跳过非 16k 音频: {file_id} ({sample_rate}Hz)")
            continue
        # 极短音频保护
        if len(audio) < int(sample_rate * 0.05):
            continue
        # ----- Step 4: 流式推理 (滑动窗口) -----
        stream = recognizer.create_stream()
        chunk_size = int(sample_rate * CHUNK_DURATION) # 1600 samples
        for i in range(0, len(audio), chunk_size):
            chunk = audio[i: i + chunk_size]
            stream.accept_waveform(sample_rate, chunk)
            # 当内部特征帧准备好时进行解码
            while recognizer.is_ready(stream):
                recognizer.decode_stream(stream)
# 通知输入结束并处理尾部

```

```

        stream.input_finished()
    while recognizer.is_ready(stream):
        recognizer.decode_stream(stream)
    # 获取最终识别结果
    result_obj = recognizer.get_result(stream)
    hyp_text_raw = (
        result_obj
        if isinstance(result_obj, str)
        else getattr(result_obj, 'text', str(result_obj))
    )
    # ----- Step 5: WER 评测 -----
    hyp_clean = normalize_tedlium_text(hyp_text_raw)
    try:
        out = jiwer.process_words(ref_clean, hyp_clean)
        curr_dist = (
            out.substitutions +
            out.deletions +
            out.insertions
        )
        curr_wer = out.wer
        curr_len = len(out.references[0]) if out.references else 0
        total_distance += curr_dist
        total_ref_words += curr_len
        file_count += 1
        print(
            f"ID: {file_id} | "
            f"WER: {curr_wer:.4f} | "
            f"Dist: {curr_dist} | "
            f"RefWords: {curr_len}"
        )
    except Exception as e:
        print(f"评测计算错误 {file_id}: {e}")
# ===== 结果汇总 =====
print("\n" + "=" * 50)
print("TEDLIUM 评测完成")
print(f"有效处理文件数: {file_count}")
print(f"忽略的过短文件数 (Words < 3): {ignored_short_count}")
print(f"未找到音频的文件数: {missing_audio_count}")
if total_ref_words > 0:
    avg_wer = total_distance / total_ref_words
    print(f"总编辑距离: {total_distance}")
    print(f"总参考单词数: {total_ref_words}")
    print("-" * 25)
    print(f"平均错词率 (Average WER): {avg_wer:.2%}")
else:
    print("未生成任何有效统计数据。")
print("=" * 50)
if __name__ == "__main__":
    main()

```

```
=====
文件名称: benchmark_Sherpa-onnx.py
=====
```

```
import sys
import os
import time
import wave
import numpy as np
import sherpa_onnx
from collections import deque
# ===== 参数配置区 =====
# LibriSpeech test-clean 数据集根目录，用于批量语音文件测试
LIBRISPEECH_DIR = "/opt/Audio_Datasets/LibriSpeech_WAV/test-clean"
# 在线流式语音识别模型所在目录
MODEL_DIR = "sherpa-onnx-streaming-zipformer-en-2023-06-21"
# 并发流数量，用于模拟同时进行的多路会议或多用户语音输入
CONCURRENT_STREAMS = 1
# 音频切片时长（秒），用于模拟实时流式输入
CHUNK_SECONDS = 0.1
# 推理设备类型："cpu" 或 "cuda"
PROVIDER = "cuda"
# =====
class WorkerSlot:
    """
    工位类：
    用于表示并发识别池中的一个处理单元，
    每个工位对应一路独立的流式语音识别会话。
    """
    def __init__(self):
        self.is_active = False          # 当前工位是否正在处理任务
        self.stream = None               # sherpa-onnx 在线识别流对象
        self.filename = None             # 当前处理的音频文件名
        self.samples = None              # 音频采样数据（float32）
        self.duration = 0.0              # 音频时长（秒）
        self.cursor = 0                  # 当前已输入的采样点位置
        self.start_time = 0.0            # 物理开始时间（用于性能统计）
        self.ttft = None                 # 首字返回时间（Time To First Token）
        self.has_output = False          # 是否已经产生过识别结果
        self.input_finished = False      # 是否已经完成音频输入
    def reset(self):
        """
        重置工位状态，用于释放资源并复用工位
        """
        self.is_active = False
        self.stream = None
        self.filename = None
        self.samples = None
        self.duration = 0.0
        self.cursor = 0
```

```

        self.start_time = 0.0
        self.ttft = None
        self.has_output = False
        self.input_finished = False
def find_wav_files(root_dir):
    """
    递归遍历目录，查找所有 WAV 音频文件
    :param root_dir: 根目录
    :return: WAV 文件路径列表
    """

    wav_files = []
    for root, dirs, files in os.walk(root_dir):
        for file in files:
            if file.lower().endswith(".wav"):
                wav_files.append(os.path.join(root, file))
    return sorted(wav_files)
def read_wav_file(filename):
    """
    读取 WAV 文件并转换为模型可用的浮点音频数据
    要求：
    - 单声道
    - 16kHz 采样率
    :param filename: WAV 文件路径
    :return: (音频采样数组，音频时长)
    """

    try:
        with wave.open(filename, "rb") as wf:
            if wf.getnchannels() != 1 or wf.getframerate() != 16000:
                return None, 0
            num_frames = wf.getnframes()
            data = wf.readframes(num_frames)
            # int16 → float32，并归一化到 [-1, 1]
            samples = np.frombuffer(data, dtype=np.int16).astype(np.float32) / 32768.0
            duration = num_frames / 16000.0
            return samples, duration
    except Exception as e:
        print(f"读取失败 {filename}: {e}")
        return None, 0
def main():
    """
    主程序入口：
    负责模型加载、并发调度、流式推理以及性能统计
    """

    # ----- 1. 数据集检查 -----
    if not os.path.exists(LIBRISPEECH_DIR):
        print(f"错误：目录不存在：{LIBRISPEECH_DIR}")
        return
    print("正在搜索音频文件...")
    all_files = find_wav_files(LIBRISPEECH_DIR)
    if not all_files:

```

```

        print("未找到 wav 文件")
        return
# 使用队列管理待处理任务
task_queue = deque(all_files)
total_files_count = len(all_files)
print(f"共发现 {total_files_count} 个音频任务。")
# ----- 2. 初始化在线识别模型 -----
print(f"加载模型: {MODEL_DIR} ({PROVIDER})...")
try:
    recognizer = sherpa_onnx.OnlineRecognizer.from_transducer(
        tokens=os.path.join(MODEL_DIR, "tokens.txt"),
        encoder=os.path.join(MODEL_DIR, "encoder-epoch-99-avg-1.onnx"),
        decoder=os.path.join(MODEL_DIR, "decoder-epoch-99-avg-1.onnx"),
        joiner=os.path.join(MODEL_DIR, "joiner-epoch-99-avg-1.onnx"),
        num_threads=1,
        sample_rate=16000,
        feature_dim=80,
        decoding_method="greedy_search",
        provider=PROVIDER,
        enable_endpoint_detection=False
    )
except Exception as e:
    print(f"模型初始化失败: {e}")
    return
# ----- 3. 初始化并发工位 -----
slots = [WorkerSlot() for _ in range(CONCURRENT_STREAMS)]
finished_results = []
sample_rate = 16000
chunk_samples = int(sample_rate * CHUNK_SECONDS)
print("\n" + "=" * 60)
print(f"开始流式识别性能测试 | 并发流数: {CONCURRENT_STREAMS}")
print("=" * 60)
start_benchmark_time = time.time()
files_processed = 0
# ===== 主调度循环 =====
while len(task_queue) > 0 or any(s.is_active for s in slots):
    # --- 4. 任务分配: 为空闲工位分配新音频 ---
    for i in range(CONCURRENT_STREAMS):
        if not slots[i].is_active and task_queue:
            next_file = task_queue.popleft()
            samples, duration = read_wav_file(next_file)
            if samples is None:
                continue
            slots[i].reset()
            slots[i].is_active = True
            slots[i].stream = recognizer.create_stream()
            slots[i].filename = next_file
            slots[i].samples = samples
            slots[i].duration = duration
            slots[i].cursor = 0

```

```

        slots[i].start_time = time.perf_counter()
# --- 5. 音频切片并输入模型（模拟实时流） ---
for s in slots:
    if s.is_active and not s.input_finished:
        start = s.cursor
        end = min(start + chunk_samples, len(s.samples))
        chunk = s.samples[start:end]
        # 对不足一个切片的尾部补零
        if len(chunk) < chunk_samples:
            padding = np.zeros(chunk_samples - len(chunk), dtype=np.float32)
            chunk = np.concatenate((chunk, padding))
        s.stream.accept_waveform(sample_rate, chunk)
        s.cursor += chunk_samples
        if start + chunk_samples >= len(s.samples):
            s.stream.input_finished()
            s.input_finished = True
# --- 6. 推理阶段 ---
ready_streams = [
    s.stream for s in slots
    if s.is_active and recognizer.is_ready(s.stream)
]
if ready_streams:
    recognizer.decode_streams(ready_streams)
# --- 7. 结果检查与性能统计 ---
current_wall_time = time.perf_counter()
for s in slots:
    if not s.is_active:
        continue
    # 首字返回时间（TTFT）计算
    if not s.has_output:
        res = recognizer.get_result(s.stream)
        text = res if isinstance(res, str) else res.text
        if text.strip():
            audio_lag = s.cursor / sample_rate
            compute_lag = current_wall_time - s.start_time
            s.ttft = audio_lag + compute_lag
            s.has_output = True
    # 任务完成判断
    if s.input_finished and not recognizer.is_ready(s.stream):
        compute_duration = current_wall_time - s.start_time
        rtf = compute_duration / s.duration
        final_ttft = s.ttft if s.ttft else (s.duration + compute_duration)
        finished_results.append({
            "filename": os.path.basename(s.filename),
            "duration": s.duration,
            "rtf": rtf,
            "ttft": final_ttft
        })
    files_processed += 1
print(

```

```

        f"[{files_processed}/{total_files_count}] "
        f"{os.path.basename(s.filename)[:25]} | "
        f"RTF={rtf:.4f}, TTFT={final_ttft:.4f}s"
    )
    s.reset()
# ===== 8. 汇总报告 =====
total_time = time.time() - start_benchmark_time
print("\n" + "=" * 60)
print("测试完成，性能统计结果如下：")
print("=" * 60)
if finished_results:
    avg_rtf = sum(r["rtf"] for r in finished_results) / len(finished_results)
    avg_ttft = sum(r["ttft"] for r in finished_results) / len(finished_results)
    print(f"总耗时: {total_time:.2f} 秒")
    print(f"处理文件数: {len(finished_results)}")
    print(f"平均 RTF: {avg_rtf:.4f}")
    print(f"平均 TTFT: {avg_ttft:.4f} 秒")
# 保存测试结果
with open("benchmark_librispeech_final.txt", "w") as f:
    f.write("Filename,Duration,RTF,TTFT\n")
    for r in finished_results:
        f.write(
            f"{r['filename']},{r['duration']:.4f}, "
            f"{r['rtf']:.4f},{r['ttft']:.4f}\n"
        )
    print("详细结果已保存至 benchmark_librispeech_final.txt")
if __name__ == "__main__":
    main()

```

```

=====
文件名称: benchmark_Sherpa-onnx_BERT.py
=====

```

```

import sys
import os
import time
import wave
import numpy as np
import sherpa_onnx
from collections import deque
import torch
from transformers import pipeline
# =====
#                               参数与路径配置区
# =====
# LibriSpeech 数据集目录，用于批量语音评测
LIBRISPEECH_DIR = "/opt/Audio_Datasets/LibriSpeech_WAV/test-clean"
# 在线流式 ASR 模型目录
ASR_MODEL_DIR = "sherpa-onnx-streaming-zipformer-en-2023-06-21"
# 标点恢复模型名称（基于 Bert 的序列标注模型）

```

```

PUNCT_MODEL_NAME = "oliverguhr/fullstop-punctuation-multilang-large"
# 并发流数量，用于模拟多路语音同时识别
CONCURRENT_STREAMS = 2
# 音频切片时长（秒），用于模拟实时语音流输入
CHUNK_SECONDS = 0.1
# 标点修正触发间隔（秒），避免过于频繁调用 Bert
BERT_INTERVAL = 0.5
# ===== 设备配置 =====
# ASR 推理设备：
#   - "cpu": 用于测试 CPU 推理性能
#   - "cuda": 用于测试 GPU 推理性能
ASR_PROVIDER = "cpu"
# Bert 标点恢复模型设备：
#   - GPU 可用时使用 GPU (device id = 0)
#   - 无 GPU 时自动回退到 CPU (device id = -1)
BERT_DEVICE_ID = 0 if torch.cuda.is_available() else -1
# =====
class PunctuationRestorer:
    """
    标点恢复模块：
    基于 Bert 的 token-classification 模型，
    对 ASR 输出的连续文本进行标点符号预测与修正。
    """
    def __init__(self, model_name, device_id):
        device_str = "GPU (cuda:0)" if device_id >= 0 else "CPU"
        print(f"正在加载 Bert 模型: {model_name} 到 {device_str}...")
        try:
            # HuggingFace pipeline:
            # - task: token-classification
            # - aggregation_strategy="none" 表示保留 token 级别结果
            # - device: -1 表示 CPU, >=0 表示 GPU ID
            self.pipe = pipeline(
                "token-classification",
                model=model_name,
                aggregation_strategy="none",
                device=device_id
            )
            print("Bert 模型加载完成。")
        except Exception as e:
            print(f"Bert 模型加载失败: {e}")
            sys.exit(1)
    def restore(self, text):
        """
        对输入文本执行标点预测。
        当前程序中主要用于模拟真实系统中的标点修正开销，
        返回文本本身，不影响性能指标计算逻辑。
        """
        if not text or not text.strip():
            return text
        try:

```



```

        _ = self.pipe(text)
        return text
    except Exception:
        return text
class WorkerSlot:
    """
    工位类（并发槽位）：
    每一个 WorkerSlot 表示一路独立的流式语音识别任务，
    用于并发调度和状态管理。
    """
    def __init__(self):
        self.is_active = False          # 是否正在处理任务
        self.stream = None               # sherpa-onnx 在线识别流
        self.filename = None             # 当前处理的音频文件名
        self.samples = None              # 音频采样数据
        self.duration = 0.0              # 音频时长（秒）
        self.cursor = 0                  # 当前已输入的采样点位置
        self.start_time = 0.0            # 物理开始时间
        self.raw_text = ""               # 当前累计识别文本
        self.ttft = None                 # 首字符上屏时间（TTFT）
        self.has_output = False           # 是否已经产生过首字
        self.input_finished = False       # 是否已完成音频输入
        self.last_punct_time = 0.0        # 上一次标点修正时间
    def reset(self):
        """
        重置工位状态，用于任务完成后的资源复用
        """
        self.is_active = False
        self.stream = None
        self.filename = None
        self.samples = None
        self.duration = 0.0
        self.cursor = 0
        self.start_time = 0.0
        self.raw_text = ""
        self.ttft = None
        self.has_output = False
        self.input_finished = False
        self.last_punct_time = 0.0
def find_wav_files(root_dir):
    """
    递归扫描目录，查找所有 WAV 文件
    """
    wav_files = []
    for root, dirs, files in os.walk(root_dir):
        for file in files:
            if file.lower().endswith(".wav"):
                wav_files.append(os.path.join(root, file))
    return sorted(wav_files)
def read_wav_file(filename):

```

```

"""
读取 WAV 文件并转换为模型可用的 float32 音频数据
要求：
    - 单声道
    - 16kHz 采样率
"""

try:
    with wave.open(filename, "rb") as wf:
        if wf.getnchannels() != 1 or wf.getframerate() != 16000:
            return None, 0
        frames = wf.readframes(wf.getnframes())
        samples = np.frombuffer(frames, dtype=np.int16).astype(np.float32) / 32768.0
        return samples, wf.getnframes() / 16000.0
except Exception:
    return None, 0

def main():
    """
    主流程：
    1. 加载数据集
    2. 初始化 ASR 与 Bert 模型
    3. 并发调度流式识别任务
    4. 统计 RTF 与 TTFT 性能指标
    """

    # ----- 数据集检查 -----
    if not os.path.exists(LIBRISPEECH_DIR):
        print("数据集不存在")
        return
    all_files = find_wav_files(LIBRISPEECH_DIR)
    if not all_files:
        print("未找到文件")
        return
    task_queue = deque(all_files)
    print(f"任务总数: {len(all_files)}")
    # ----- 初始化 ASR -----
    print(f"加载 ASR 模型到: {ASR_PROVIDER}")
    try:
        recognizer = sherpa_onnx.OnlineRecognizer.from_transducer(
            tokens=os.path.join(ASR_MODEL_DIR, "tokens.txt"),
            encoder=os.path.join(ASR_MODEL_DIR, "encoder-epoch-99-avg-1.onnx"),
            decoder=os.path.join(ASR_MODEL_DIR, "decoder-epoch-99-avg-1.onnx"),
            joiner=os.path.join(ASR_MODEL_DIR, "joiner-epoch-99-avg-1.onnx"),
            num_threads=1,
            sample_rate=16000,
            feature_dim=80,
            decoding_method="greedy_search",
            provider=ASR_PROVIDER,
            enable_endpoint_detection=False
        )
    except Exception as e:
        print(e)

```

```

    return
# ----- 初始化 Bert -----
bert_restorer = PunctuationRestorer(PUNCT_MODEL_NAME, BERT_DEVICE_ID)
# ----- 初始化并发工位 -----
slots = [WorkerSlot() for _ in range(CONCURRENT_STREAMS)]
finished_results = []
chunk_samples = int(16000 * CHUNK_SECONDS)
print(f"\n开始测试 | ASR: {ASR_PROVIDER} | Bert: {'GPU' if BERT_DEVICE_ID >= 0 else 'CPU'}")
print(f"并发数: {CONCURRENT_STREAMS} | ASR 切片: {CHUNK_SECONDS}s")
# ----- 主调度循环 -----
while task_queue or any(s.is_active for s in slots):
    current_wall_time = time.perf_counter()
    # 分配任务到空闲工位
    for i in range(CONCURRENT_STREAMS):
        if not slots[i].is_active and task_queue:
            fname = task_queue.popleft()
            samples, duration = read_wav_file(fname)
            if samples is None:
                continue
            s = slots[i]
            s.reset()
            s.is_active = True
            s.stream = recognizer.create_stream()
            s.filename = fname
            s.samples = samples
            s.duration = duration
            s.start_time = current_wall_time
            s.last_punct_time = current_wall_time
    # 音频切片并输入 ASR
    for s in slots:
        if s.is_active and not s.input_finished:
            start = s.cursor
            end = min(start + chunk_samples, len(s.samples))
            chunk = s.samples[start:end]
            if len(chunk) < chunk_samples:
                chunk = np.concatenate(
                    (chunk, np.zeros(chunk_samples - len(chunk), dtype=np.float32))
                )
            s.stream.accept_waveform(16000, np.ascontiguousarray(chunk))
            s.cursor += chunk_samples
            if start + chunk_samples >= len(s.samples):
                s.stream.input_finished()
                s.input_finished = True
    # ASR 解码
    ready = [s.stream for s in slots if s.is_active and recognizer.is_ready(s.stream)]
    if ready:
        recognizer.decode_streams(ready)
    loop_time = time.perf_counter()
    # 结果处理与指标计算
    for s in slots:

```

```

        if not s.is_active:
            continue
        res = recognizer.get_result(s.stream)
        text = res if isinstance(res, str) else res.text
        text = text.strip()
        # 首字上屏时间 (TTFT) 计算
        if text != s.raw_text:
            s.raw_text = text
            if not s.has_output and text:
                audio_elapsed = s.cursor / 16000.0
                latency = loop_time - s.start_time
                s.ttft = audio_elapsed + latency
                s.has_output = True
        # 周期性执行 Bert 标点修正
        if loop_time - s.last_punct_time >= BERT_INTERVAL and s.raw_text:
            _ = bert_restorer.restore(s.raw_text)
            s.last_punct_time = time.perf_counter()
        # 任务完成处理
        if s.input_finished and not recognizer.is_ready(s.stream):
            if s.raw_text:
                _ = bert_restorer.restore(s.raw_text)
            process_duration = time.perf_counter() - s.start_time
            rtf = process_duration / s.duration
            ttft_val = s.ttft if s.ttft else (s.duration + process_duration)
            finished_results.append({
                "filename": os.path.basename(s.filename),
                "rtf": rtf,
                "ttft": ttft_val
            })
            print(f"[完成] RTF: {rtf:.4f} | TTFT: {ttft_val:.4f}s")
            s.reset()
    # ----- 汇总统计 -----
    if finished_results:
        avg_rtf = sum(r["rtf"] for r in finished_results) / len(finished_results)
        avg_ttft = sum(r["ttft"] for r in finished_results) / len(finished_results)
        print("\n" + "=" * 60)
        print(f"平均 RTF: {avg_rtf:.4f}")
        print(f"平均 TTFT: {avg_ttft:.4f} s")
        print("=" * 60)
if __name__ == "__main__":
    main()

```